

Minimize to Maximize: Provable Utility and Privacy

Xinyu Zhou¹ Wenfei Fan^{1,2} Zihuan Xu¹

¹Shenzhen Institute of Computing Sciences, China ²University of Edinburgh, UK
zhouxinyu@sics.ac.cn, wenfei@inf.ed.ac.uk, xuzihuan@sics.ac.cn

ABSTRACT

Training data minimization for relational ML asks which inputs a task needs and how much information about them should be retained. We formulate this as a model-aware transformation of a training relation and its schema, minimizing information leakage subject to an accuracy target, and show that the problem is NP-hard. We propose M2M (Minimize to Maximize), which jointly learns entropy-weighted input gates, noisy linear aggregation, and the classifier under one accuracy constraint. Its key idea is a privacy-driven trace objective: minimizing projected covariance trace favors low-variance representations compatible with task accuracy, minimizes an information-leakage bound for fixed masked data, output dimension and nonzero release noise, and under orthonormal projection, maximizes expected back-projection error. For fixed feasible transformations, we further derive a lower bound on privacy gain under the empirical input distribution and develop an accuracy-guided optimization algorithm. The learned transformation can be reused on subsequent records to reduce input acquisition and data retention. Using real-life and synthetic datasets, we show that M2M reduces normalized reconstruction risk against feature-only learned attackers by 32.25% on average over the best baseline under the same utility constraint, and scales to one million examples.

PVLDB Reference Format:

Xinyu Zhou, Wenfei Fan, and Zihuan Xu. Minimize to Maximize: Provable Utility and Privacy. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/xyzhou055/m2m-paper>.

1 INTRODUCTION

Machine learning (ML) on relational data raises a data-management question: *what information does a prediction task actually need its training relation to retain?* A relation may contain attributes that contribute little to prediction, and precise values whose coarser representations suffice for the task. Retaining such inputs and details exposes raw information to reconstruction, attribute inference [57, 62, 67] and linkage attacks [31, 65]. Reducing this exposure therefore calls for minimizing the training data itself, rather than only protecting its computation or learned model.

We study *training data minimization*. Given a relation $\mathcal{D}$ of schema $\mathcal{R}$ and a target classifier $\mathcal{M}$, we compute a transformed

relation $\mathcal{D}_M$ of schema $\mathcal{R}_M$ that reveals less about the raw features while meeting a prescribed accuracy target. The transformation changes both schema and instance: it removes unnecessary inputs and represents retained ones with less revealing aggregates. The goal is to retain only the information needed by the target task.

Example 1: Consider cardiovascular-risk prediction from demographic attributes (Age, Weight, Height, Gender), clinical measurements such as three blood-pressure readings (BP[1], BP[2], BP[3]) and Cholesterol, and lifestyle attributes (Smoking, Alcohol Intake).

Data minimization combines two operations. *Feature removal* may drop Height when it is unnecessary for prediction, thus eliminating its future acquisition. *Feature aggregation* may instead replace the three blood-pressure readings by $\text{AverageBP} = 0.2 \cdot \text{BP}[1] + 0.3 \cdot \text{BP}[2] + 0.5 \cdot \text{BP}[3]$, reducing the information retained while still requiring the original readings to compute the aggregate. These decisions are coupled: for example, pruning Weight may hurt accuracy if it interacts with Cholesterol. Thus removal and aggregation must be jointly optimized under the accuracy target. $\square$

VLDB relevance. Training data minimization operates directly over database schemas and instances, determining which attributes to retain and how to represent them before ML training. It is a model-aware data transformation problem at the intersection of data management for ML and data privacy, both within the VLDB scope.

Existing approaches address only parts of this problem. Feature selection [58, 64, 68] chooses predictive inputs, but retained raw values may still disclose more than the task needs [19]. Vertical minimization uses feature selection or generalization [19, 22, 56], while information bottleneck and supervised compression [3, 42, 54, 59] learn compact task-aware representations. However, a compact representation may still depend on every original input, and thus does not identify attributes that can be omitted from subsequent acquisition. Differential privacy [2, 6], secure computation [33, 35], and federated learning [12, 17, 43] protect aspects of training, computation or release, while tuple sampling [10, 44, 45, 63] reduces records rather than information retained per record.

The difficulty is that removal and aggregation are coupled: removing an input changes the aggregates available to the model, and aggregation changes which inputs are needed to meet the accuracy target. Optimizing either alone may retain unnecessary data or discard useful signal. This motivates a unified objective: *jointly minimize input dependence and retained information subject to task utility*.

Our approach. We propose M2M (Minimize to Maximize), which jointly learns a minimized representation and classifier under one accuracy constraint. Entropy-weighted gates reduce dependence on unnecessary inputs, while noisy linear aggregation reduces information retained from those that remain.

Its key technical contribution is a *privacy-driven trace objective*. M2M minimizes projected covariance trace, favoring low-variance directions compatible with task accuracy, in contrast to

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

principal component analysis (PCA) [27], which preserves high-variance directions. For fixed masked data, output dimension and nonzero Gaussian release noise, trace minimization minimizes an information-leakage bound; under orthonormal projection, it also maximizes expected squared orthogonal back-projection error.

We proceed from hardness to optimization and guarantees. We show that even feature removal is NP-hard, derive tractable objectives for removal and aggregation, and combine them under a common accuracy constraint. For fixed feasible transformations with nonzero Gaussian release noise, we derive a lower bound on privacy gain under the empirical input distribution, while the accuracy target bounds utility loss. An accuracy-guided algorithm computes utility-feasible solutions in one training run.

Once learned, the transformation can also be reused on subsequent records: inputs no longer required can be dropped from acquisition, while retained inputs can be stored or released only via minimized aggregates. Thus M2M reduces data exposure at the source, rather than only after full data have already been collected.

Contributions & Organization. After reviewing background and related work (Section 2), we make the following contributions.

(1) *Formalization and complexity (Section 3).* We formulate training data minimization as a model-aware transformation of a relation and its schema, maximizing privacy gain subject to target model accuracy. We show that even its feature-removal case is NP-hard.

(2) *Model-aware minimization (Sections 4-6).* We propose M2M, a two-stage framework (Section 4) that combines (a) entropy-weighted input removal through trainable gates (Section 5) and (b) privacy-driven noisy linear aggregation that minimizes projected covariance trace (Section 6), under one accuracy constraint. For fixed centered masked data, output dimension and nonzero Gaussian release noise, the trace objective minimizes an information-leakage bound and, under orthonormal projection, is equivalent to maximizing expected squared back-projection error.

(3) *Algorithm and guarantees (Section 7).* We combine the gating and trace penalties to jointly learn the gates, aggregation and classifier under an accuracy constraint. For fixed feasible transformations with nonzero Gaussian noise added to the released representation, we derive a lower bound on privacy gain under the empirical input distribution; the prescribed accuracy target bounds their utility loss. We develop an accuracy-guided algorithm that uses differentiable model loss for parameter updates and validation accuracy to adjust the privacy-utility balance in one training run.

(4) *Experimental study (Section 8).* Using five real-life datasets and synthetic data, we evaluate privacy, utility and scalability. (a) Under the same utility threshold, M2M reduces normalized reconstruction risk (nrr) against feature-only learned attackers by 32.25% on average over the strongest tested baseline, up to 44.46%; the advantage persists across the tested attacker and classifier choices. (b) Component ablations support gating and the trace objective within the same linear family: adding gates reduces mean nrr from 14.28% to 12.27%; omitting the trace term increases it to 24.57%. (c) M2M scales to 1M examples and 200 features in 1484s. A feature-level case study shows which original attributes are removed or partially retained.

We discuss future work in Section 9, and defer proofs to [1].

2 BACKGROUND AND PREVIOUS WORK

This section reviews the basic notations of data minimization, the concepts underlying privacy and utility, and relevant prior work.

Relations. Consider a relation schema $\mathcal{R} = (A_1, \dots, A_n, A_y)$ with feature attributes $A_1, \dots, A_n$ and label attribute A_y . A dataset $\mathcal{D}$ is a relation of $\mathcal{R}$. For multi-relation sources, $\mathcal{D}$ is the training relation obtained via standard joins and feature construction. An input feature is a column of this relation after preprocessing; an original source attribute may yield several columns, such as one-hot indicators.

Data distribution. We focus on classification tasks, where an ML instance consists of a feature vector $x \in \mathcal{X} \subseteq \mathbb{R}^n$ and a label $y \in \mathcal{Y}$. Let $X = (X_1, \dots, X_n)$ be the feature random vector, where X_i corresponds to attribute A_i , and let Y be the label random variable. We assume $(X, Y) \sim P$ for an unknown joint distribution P over $\mathcal{X} \times \mathcal{Y}$. The dataset $\mathcal{D}$ consists of i.i.d. realizations (x, y) of (X, Y) .

Accuracy. For model family $\mathcal{M}$ and a dataset $\mathcal{D}$, let $\mathcal{M}(\mathcal{D}) : \mathcal{X} \rightarrow \mathcal{Y}$ be trained on $\mathcal{D}$. Its *accuracy* under distribution P is defined as $\text{acc}(\mathcal{M}(\mathcal{D})) = \mathbb{E}_{(x,y) \sim P} [\mathbb{1}(\mathcal{M}(\mathcal{D})(x) = y)]$.

Entropy and mutual information. Given a random variable X over a finite data universe $\mathcal{X}$ with probability mass function $P_X(x)$, the *Shannon entropy* of X , which quantifies the inherent uncertainty of X , is defined as $H(X) = -\sum_{x \in \mathcal{X}} P_X(x) \log P_X(x)$.

Moreover, given two random variables X and Y with joint probability mass function $P_{X,Y}(x, y)$ and conditional probability mass function $P_{Y|X}(y | x)$, the *mutual information* between X and Y is

$$I(X; Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P_{X,Y}(x, y) \log \left(\frac{P_{Y|X}(y | x)}{P_Y(y)} \right).$$

Mutual information is equivalent to the uncertainty reduction, which is written as $I(X; Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$. Unless otherwise stated, all logarithms are natural.

Data minimization. We focus on *vertical data minimization* for ML, which seeks a transformation ϕ that maps $\mathcal{D} = (\mathcal{D}_x, \mathcal{D}_y)$ to $\mathcal{D}_M = (\phi(\mathcal{D}_x), \mathcal{D}_y)$, reducing feature dimensionality or granularity. The objective is to *maximize* privacy gain and *minimize* utility loss: the minimized data should reveal as little as possible about the raw features while preserving downstream model accuracy.

Privacy. Given a transformation ϕ , denote by $O = \phi(X)$ the *minimized representation* of the feature vector X . We measure *privacy* by $H(X | O)$, i.e., the uncertainty about the raw features after observing O ; for a fixed distribution, a larger value means that less information about X is revealed. We consider the feature-only setting, where the recipient observes O . Following the base setting of [56], our attack evaluation assumes that the recipient has auxiliary examples, but does not observe the labels or the transformation parameters. If the labels Y retained for supervised learning are also revealed, they become additional side information, and privacy is measured by $H(X | O, Y)$. We also analyze this stronger setting and derive a corresponding lower bound in Section 7.

Utility. Given an ML algorithm $\mathcal{M}$ and a minimized dataset $\mathcal{D}_M$, we measure *utility* by $\text{acc}(\mathcal{M}(\mathcal{D}_M))$, i.e., the predictive accuracy of the classification model trained on $\mathcal{D}_M$.

Example 2: Continuing Example 1, the patient’s raw readings BP = [90, 105, 97] may reveal sensitive health conditions. Since

AverageBP = 98 can arise from many raw vectors, e.g., [95, 105, 95] and [90, 100, 100], the adversary’s uncertainty $H(\text{BP} \mid \text{AverageBP})$ remains high. Meanwhile, if the prediction depends on AverageBP and other attributes, the aggregate retains the signal needed to train $\mathcal{M}$ without exposing the raw vector, thus preserving utility. $\square$

Related work. Prior work on data minimization (DM) can be broadly categorized into the following two directions:

(1) *Vertical DM* reduces the dimension or granularity of data features. Feature selection [7, 61, 64] retains task-relevant attributes. [56] proposes a vertical data minimization framework for feature generalization based on the PGini criterion, penalizing decision tree regions that are predictive of personal attributes. Similarly, [22] prunes decision trees based on data anonymization metrics. However, both approaches are model-dependent and rely on tree-based architecture for data minimization. [19] proposes an individualized data minimization framework that selects different feature subsets for different individuals. It shows that a substantial reduction in collected features does not necessarily meet users’ privacy expectations, since reconstruction and re-identification risks may remain high. In another line of work, [53] formulates data minimization for correlated sensor streams as a two-player game and develops greedy algorithms for practical optimization. [9] studies data minimization in recommendation systems, analyzing its effects on neighborhood-based and matrix-factorization recommenders.

(2) *Horizontal DM* reduces collected data points. Data pruning [48, 55] and coreset selection [10, 44, 45] remove redundant or harmful examples while retaining a compact training subset. Such curation can improve generalization, stability and performance [14, 47, 66]. For example, [52] adaptively monitors model performance to determine when data collection can stop, thus minimizing the number of collected examples. Rather than proposing a minimization method, [50] measures model instability under data imputation to audit the data-minimization compliance of black-box models.

Privacy and representation methods. Differential privacy [2, 6, 32] protects data during training or release, while synthetic data [5, 28, 34] replaces raw data with generated samples. Information bottleneck methods learn compact representations that preserve task-relevant information [3, 4, 59]; privacy funnel and privacy preserving representation learning limit the disclosure of sensitive variables [16, 40]; and supervised compression learns task-aware representations for storage or transmission [42, 54].

This work differs from prior work in the following. (1) *Minimization object*: M2M transforms a materialized training relation, jointly learning the transformation and task model, identifying inputs no longer needed by the learned transformation and producing a compact schema. (2) *Minimization operations*: Existing vertical DM methods typically perform selection or generalization alone [19, 22, 56]. Our trace objective has a linear Gaussian bottleneck interpretation (see Section 6.3). M2M jointly learns entropy-weighted input removal and trace-based aggregation under a common accuracy constraint, reducing both dimensionality and information retained per tuple, rather than the number of tuples. (3) *Privacy and utility*: Existing DM methods often rely on heuristic selection criteria [19, 56], while privacy-funnel methods protect designated sensitive variables under information-disclosure con-

straints [40]. M2M instead minimizes leakage of the raw feature vector subject to downstream accuracy, connects the transformation to orthogonal back-projection error, and provides privacy-utility bounds for fixed feasible transformations with positive release noise.

3 PROBLEM AND COMPLEXITY

This section formulates the data minimization problem (Section 3.1), and proves the intractability of its feature-removal case (Section 3.2).

3.1 The Data Minimization Problem

We start with notions of minimization, utility loss and privacy gain.

Vertical minimization. We focus on *vertical* and *global* data minimization. Given a dataset $\mathcal{D} = (\mathcal{D}_x, \mathcal{D}_y)$ with feature vectors $\mathcal{D}_x = (x_1, \dots, x_N)$ and labels $\mathcal{D}_y$, a possibly randomized operation $\phi : \mathcal{X} \rightarrow \mathcal{X}_M$ is applied uniformly to every instance, transforming only the feature space and leaving the labels unchanged:

$$\phi(\mathcal{D}_x) = (\phi(x_1), \dots, \phi(x_N)).$$

Denote by $\mathcal{D}_M = (\phi(\mathcal{D}_x), \mathcal{D}_y)$ the resulting minimized dataset. The operation is vertical because ϕ removes or aggregates attributes, reducing the dimensionality or granularity of each feature vector while keeping the tuple set and labels fixed.

Utility loss. Vertical minimization may incur a utility loss by reducing predictive accuracy. For an original dataset $\mathcal{D}$ and its minimized version $\mathcal{D}_M$, this *utility loss* is $\text{acc}(\mathcal{M}(\mathcal{D})) - \text{acc}(\mathcal{M}(\mathcal{D}_M))$.

Privacy gain. Meanwhile, minimization can yield a privacy gain by limiting the revealed raw feature information. For $O = \phi(X)$, this gain is $H(X \mid O) - H(X \mid X) = H(X \mid O)$, since $H(X \mid X) = 0$.

Remark. For a fixed original dataset, $H(X)$ is constant. Hence maximizing privacy gain $H(X \mid O)$ is equivalent to minimizing the mutual information $I(X; O) = H(X) - H(X \mid O)$, i.e., reducing how much the minimized features O reveal about the raw input X .

We optimize over a prescribed class, i.e., the allowed ways to remove or aggregate features. We prove hardness even for coordinate projections, which only remove features (see Section 3.2).

Problem. We next state the data minimization problem for ML, referred to as DMP_ML. It is a bi-criteria optimization problem.

- *Input*: Schema $\mathcal{R}$, dataset $\mathcal{D}$ of $\mathcal{R}$, model $\mathcal{M}$ to train, and an admissible class of global transformations.
- *Output*: A minimized dataset $\mathcal{D}_M$ of $\mathcal{D}$, with schema $\mathcal{R}_M$.
- *Objective*: Minimize utility loss $\text{acc}(\mathcal{M}(\mathcal{D})) - \text{acc}(\mathcal{M}(\mathcal{D}_M))$ and maximize privacy gain $H(X \mid O)$.

Minimization transforms both the instance and its schema: feature removal deletes attributes, while aggregation replaces them with new ones. Thus, DMP_ML maps $(\mathcal{R}, \mathcal{D})$ to $(\mathcal{R}_M, \mathcal{D}_M)$, making it a model-aware schema and data transformation problem rather than merely a training-time feature-selection task.

Constrained formulation. The model trained on $\mathcal{D}_M$ must achieve a target accuracy τ . Given a transformation ϕ from the prescribed class and $O = \phi(X)$, DMP_ML is then:

$$\max_{\phi} H(X \mid O) \Leftrightarrow \min_{\phi} I(X; O) \quad \text{s.t.} \quad \text{acc}(\mathcal{M}(\mathcal{D}_M)) \geq \tau. \quad (1)$$

In the optimization and analysis below, we treat X as a random draw from the empirical distribution of the training features $\mathcal{D}_x$.

3.2 Complexity

We prove hardness for the special case of DMP_ML in which ϕ is restricted to feature removal. The decision version of this special case is to decide, given a dataset $\mathcal{D}$ with schema $\mathcal{R}$, an ML model $\mathcal{M}$ to be trained, a target accuracy τ and a privacy bound ρ , whether there exists a transformed dataset $\mathcal{D}_M$ with a reduced schema $\mathcal{R}_M$ such that $\text{acc}(\mathcal{M}(\mathcal{D}_M)) \geq \tau$ and $H(X | O) \geq \rho$, where $O = \phi(X)$.

Theorem 1 shows that the feature-removal case is NP-hard.

Theorem 1: *The feature-removal case of DMP_ML is NP-hard.* $\square$

Proof sketch: We prove by reduction from the NP-complete Set Cover problem (cf. [20]). Given a universe U , a collection $\mathcal{S}$ of subsets of U , and a budget k , Set Cover asks whether at most k subsets can cover all elements of U . We construct a feature-removal instance with one feature per subset and two groups of tuples: (1) task tuples make the accuracy bound hold if and only if (iff) the retained features correspond to subsets covering U ; and (2) auxiliary tuples make the privacy bound hold iff at most k features are retained. Thus, the constructed instance is feasible iff the Set Cover instance admits a cover of size at most k (see [1] for details). $\square$

4 M2M: A MINIMIZATION FRAMEWORK

This section introduces M2M, our framework for data minimization.

Given Theorem 1, directly solving the optimization problem in Equation (1) is hard. To overcome this, we introduce M2M, which takes a schema $\mathcal{R}$, a dataset $\mathcal{D}$ and a target model $\mathcal{M}$ as input, and returns a minimized dataset $\mathcal{D}_M$ as output. The objective is to reduce the information leakage from X to O while ensuring that $\mathcal{M}$ trained on $\mathcal{D}_M$ still satisfies the utility constraint. M2M learns the transformation and task model from available training data; once learned, it can also guide subsequent data collection (see Section 5.2).

As shown in Figure 1, M2M has two stages: *feature removal*, which prunes unnecessary input features, followed by *linear feature aggregation* to further compress the data into an interpretable lower-dimensional representation. As will be seen in Section 7, for fixed feasible transformations with positive release noise, M2M provides an upper bound on utility loss and a lower bound on privacy gain.

(1) Feature removal. The first stage removes task-irrelevant features. It learns a binary gate $g \in \{0, 1\}^n$ on the raw input x , yielding

$$z = g \odot x = (g_1 x_1, \dots, g_n x_n).$$

A gate $g_i = 0$ removes feature x_i from the minimized representation. To make removal privacy-aware, M2M weights retained features by marginal entropy, penalizing more revealing attributes.

This approach differs from L_1 -regularized minimization methods [19]: rather than shrinking parameters, M2M learns explicit gates for feature removal, and thus avoids the parameter-shrinkage behavior that can hurt neural models in practice [18, 51].

(2) Feature aggregation. The second stage maps the masked features $z \in \mathbb{R}^n$ into a lower-dimensional representation $o \in \mathbb{R}^k$:

$$o = W^T z + \epsilon,$$

where $W \in \mathbb{R}^{n \times k}$ is learnable and ϵ is noise. This aggregation further reduces reconstruction risk left by feature removal, while remaining interpretable as linear combinations of raw attributes.

The goal is privacy-driven trace minimization. M2M minimizes the trace of the projected covariance, favoring low-variance direc-

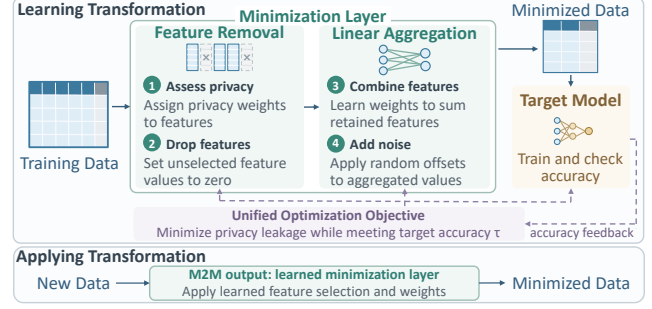


Figure 1: The architecture of the M2M framework.

tions rather than the high-variance directions preserved by principal component analysis (PCA) [27]. We show that this trace objective controls an upper bound on information leakage under positive release noise and, for fixed masked features, maximizes expected squared orthogonal back-projection error.

Unified objective. We combine feature removal and aggregation into a single constrained optimization problem. The objective minimizes a weighted sum of privacy penalties (entropy-weighted gating for removal and trace minimization for aggregation), subject to a utility constraint on the model trained over the minimized data.

Example 3: Continuing with Example 1, suppose M2M learns that Height is unnecessary for cardiovascular-risk prediction. Feature removal sets its gate to $g_i = 0$, so $z_i = g_i \cdot \text{Height} = 0$ regardless of the patient’s actual height. Since Height is blocked from the training representation, it need not be collected for subsequent records.

After removal, M2M aggregates the retained features. Suppose one column of W assigns weights $(1/\sqrt{3}, 1/\sqrt{3}, 1/\sqrt{3})$ to the three blood-pressure readings and zero weight to all other inputs. The output feature is $o_j = \frac{\text{BP}[1] + \text{BP}[2] + \text{BP}[3]}{\sqrt{3}} + \epsilon_j$, which is a noisy blood-pressure aggregate, with ϵ_j denoting the additive noise. $\square$

We detail the two stages in Sections 5 and 6. Unifying the two stages, we develop our data minimization algorithm in Section 7.

5 FEATURE REMOVAL

This section presents feature removal, the first stage of M2M. It learns binary gates to prune task-irrelevant input features and can reduce subsequent data collection. We first upper-bound joint privacy leakage by marginal entropies, yielding a weighted minimization problem (Section 5.1), and then use a differentiable sparse relaxation for efficient gradient-based optimization (Section 5.2).

5.1 Problem Reformulation

We start with a notion of feature gating.

Feature gating. To frame feature removal as an ML problem, we introduce a binary gating function $g \in \{0, 1\}^n$ with each coordinate being either 0 or 1. The output of the feature removal layer is the elementwise product $Z = g \odot X$. Because the mapping from X to Z is deterministic, the conditional entropy $H(Z|X) = 0$. Therefore, minimizing the mutual information $I(X; Z)$ is equivalent to minimizing the entropy of the gated features $H(Z) = H(g \odot X)$.

Thus, the objective in Equation (1) (Section 3) can be rewritten as

$$\min_{g \in \{0, 1\}^n} H(g \odot X) \quad \text{s.t.} \quad \text{acc}(\mathcal{M}(g \odot \mathcal{D}_x, \mathcal{D}_y)) \geq \tau,$$

where $\odot$ denotes the elementwise product.

Example 4: Recall Example 1. Consider a patient record (Height, Age, Smoking, BP). A binary gate produces $(g_1 \cdot \text{Height}, g_2 \cdot \text{Age}, g_3 \cdot \text{Smoking}, g_4 \cdot \text{BP})$, $g_i \in \{0, 1\}$. If Height is task-irrelevant, setting $g_1 \cdot \text{Height} = 0$ removes it from training, so it need not be collected at the source for subsequent records. $\square$

Marginal entropy bound. Directly optimizing over the joint entropy $H(g \odot X)$ in the objective is often challenging since estimating the joint distribution of high dimensional data is computationally costly. Thus we utilize an information-theoretic property [13] that upper bounds joint entropy by the sum of marginal entropies, written as

$$H(g \odot X) \leq \sum_{i=1}^n H(g_i \cdot X_i) = \sum_{i=1}^n g_i H(X_i).$$

For each feature X_i , denote by w_i an estimate of its marginal entropy $H(X_i)$. This yields the following weighted minimization:

$$\min_{g \in \{0,1\}^n} \sum_{i=1}^n g_i w_i \quad \text{s.t.} \quad \text{acc}(\mathcal{M}_{\theta,g}) \geq \tau. \quad (2)$$

Here $\mathcal{M}_{\theta,g}$ denotes the predictor with parameters θ and gates g .

Remark. (1) We estimate w_i from the training split using empirical marginal entropy for discrete data and empirical variance for all features in mixed-type data after scaling them to $[0, 1]$. For mixed-type data, these variance weights serve as a heuristic retention penalty. (2) Larger w_i more strongly penalizes retention and can encode domain-specific privacy preferences for highly sensitive attributes.

5.2 Differentiable Relaxation

Although Equation (2) replaces the joint entropy objective with a weighted sum of marginal entropies, the problem remains intractable. The gate $g \in \{0, 1\}^n$ induces an exponential search space. Indeed, with $w_i = 1$, the Set Cover construction in Theorem 1 makes the accuracy constraint hold iff the retained features cover the universe; minimizing the objective then finds a minimum cover. Moreover, the discrete gate is not compatible with gradient-based optimization, making it impractical for high-dimensional ML tasks.

In light of this, we adopt the hard-concrete relaxation [38] to avoid optimizing the binary gates directly. It assigns each attribute A_i a learnable score s_i and converts this score into a random gate $g_i \in [0, 1]$ during training. A larger s_i makes the gate more likely to be active. More specifically, for $u_i \sim \mathcal{U}(0, 1)$, we set

$$g_i = \text{clip}_{[0,1]} \left(\text{sigmoid} \left(\frac{\log u_i - \log(1 - u_i) + s_i}{\beta} \right) (\zeta - \gamma) + \gamma \right),$$

where $\text{clip}_{[0,1]}$ truncates its input to $[0, 1]$, with $\beta > 0$, $\gamma < 0$, and $\zeta > 1$ set as in the standard hard-concrete construction [38]. Its continuous values allow s_i and θ to be jointly optimized by gradient descent, while exact zeros and ones permit discrete gating decisions. Since g_i is active with probability $p_i(s_i) = \Pr(g_i > 0) = \text{sigmoid}(s_i - \beta \log(-\gamma/\zeta))$, the expected weighted penalty for retaining A_i is $w_i p_i(s_i)$. Summing over attributes gives

$$\min_{s \in \mathbb{R}^n, \theta \in \mathbb{R}^p} \sum_{i=1}^n w_i p_i(s_i) \quad \text{s.t.} \quad \text{acc}(\mathcal{M}_{\theta,s}) \geq \tau,$$

where $\mathcal{M}_{\theta,s}$ denotes the predictor with parameters θ and scores s .

At each training step, we sample g_i from the current score s_i . For utility evaluation, we instead use the deterministic gate

$$\widehat{g}_i(s_i) = \text{clip}_{[0,1]} (\text{sigmoid}(s_i)(\zeta - \gamma) + \gamma).$$

After training, input feature i is removed if $\widehat{g}_i(s_i) = 0$ and retained otherwise. An original attribute can be omitted from future acquisition only if none of the retained input features depends on it. For example, under per-attribute one-hot encoding, the attribute can be dropped only when all of its indicators are removed; retaining any indicator still requires the original attribute value.

Example 5: Continuing Example 4, suppose Height and Age have weights $w_1 = 2w$ and $w_2 = w$, with $p_1(s_1) = p_2(s_2) = p$. While hard-concrete samples may lie in $(0, 1)$, their expected active-gate penalties are $2wp$ and wp , respectively. Thus, Height is more strongly penalized unless needed to satisfy the accuracy constraint. $\square$

Remark. Feature-removal turns vertical minimization into a trainable gating problem. For discrete inputs, the marginal-entropy bound induces a weighted sparsity penalty, while the hard-concrete relaxation enables gradient-based removal. In practice, M2M can reduce subsequent data acquisition, penalize more revealing features, and learn minimization jointly with the downstream task.

6 LINEAR AGGREGATION

This section develops the aggregation stage of M2M. We first define the linear aggregation layer and its noisy lower-dimensional output (Section 6.1). We then derive an entropy-based privacy bound (Section 6.2), obtain a trace surrogate under orthonormal projections (Section 6.3), and prove that trace minimization maximizes expected squared back-projection error for fixed masked data (Section 6.4).

6.1 Linear Aggregation

We restrict our approach to linear aggregation, which is expressive enough to combine correlated retained features while keeping the transformation simple and auditable. Each constructed feature is an explicit linear combination of the raw inputs, allowing practitioners to trace exactly which inputs contribute and by how much. This interpretability is important for data minimization, where the retained information should be directly inspectable.

Given the masked feature vector $z \in \mathbb{R}^n$ produced by the removal layer, we further compress it into k aggregated features $o \in \mathbb{R}^k$ using a learnable projection matrix $W \in \mathbb{R}^{n \times k}$:

$$o = W^T z + \epsilon,$$

where ϵ has i.i.d. Gaussian entries, i.e., $\epsilon \sim \mathcal{N}(0, \sigma^2 I_k)$. Equivalently, each output feature $o_i \in \mathbb{R}$ for $i \in [k]$ is a noisy linear combination of the elements in z , defined by the dot product $o_i = W_{:,i}^T z + \epsilon_i$.

Note that the additive Gaussian noise ϵ has two roles:

(1) Inspired by the DP Gaussian mechanism [15], the noise helps defend against reconstruction: instead of revealing the exact projection $o = W^T z$, Gaussian noise smooths nearby outputs and makes them harder to distinguish. We use it as an additional privacy enhancement, without claiming formal DP guarantees.

(2) The noise also avoids entropy degeneracy in deterministic neural representations [4, 30]. A noiseless projection $O = W^T Z$ may preserve all distinct empirical records and hence their discrete entropy. A positive-noise Gaussian channel smooths the output into a continuous distribution with well-defined differential entropy [3, 21, 30].

Remark. For a fixed gate mask retaining d coordinates, let $\widehat{z} \in \mathbb{R}^d$

contain the retained coordinates of z , and $\widehat{W} \in \mathbb{R}^{d \times k}$ contain the corresponding rows of W . Since all other coordinates of z are zero, one can derive that $o = W^T z + \epsilon = \widehat{W}^T \widehat{z} + \epsilon$. Hence, W can remain $n \times k$ throughout training without changing the output.

Example 6: Continuing Example 4, suppose Height has been pruned, leaving $z = (0, \text{Age}, \text{Smoking}, \text{BP}[1], \text{BP}[2], \text{BP}[3]) \in \mathbb{R}^6$. A learnable projection W , together with Gaussian noise $\epsilon = (\epsilon_1, \epsilon_2)$, maps z to two aggregated features: $\text{LifestyleIndex} = \frac{\text{Age} + \text{Smoking}}{\sqrt{2}} + \epsilon_1$, and $\text{BPAggregate} = \frac{\text{BP}[1] + \text{BP}[2] + \text{BP}[3]}{\sqrt{3}} + \epsilon_2$. Thus, the retained features are compressed into two noisy aggregates. $\square$

6.2 Entropy Minimization in Linear Space

Under our linear aggregation formulation framework, the mutual information between the input X and output O can be written as:

$$I(X; O) = h(O) - h(O|X) = h(O) - h(\epsilon) = h(O) - \frac{k}{2} \ln(2\pi e \sigma^2),$$

where $h(\cdot)$ is the differential entropy: $h(Y) = -\int f(y) \log f(y) dy$ for a continuous random variable Y with density $f(y)$.

Since the noise entropy $h(\epsilon)$ is a fixed constant, minimizing the mutual information is equivalent to minimizing the differential entropy $h(O)$. To characterize this, let $\bar{z} = \frac{1}{N} \sum_{i=1}^N z_i$ and $\Sigma = \frac{1}{N} \sum_{i=1}^N (z_i - \bar{z})(z_i - \bar{z})^T$ be the empirical covariance matrix of the selected features Z . Due to the linearity of the transformation, the covariance matrix of the output O is

$$K_O = \text{Cov}(O) = W^T \Sigma W + \sigma^2 I_k.$$

Directly minimizing the exact entropy of this representation O can be prohibitively costly. Instead, we aim to minimize a tractable upper bound, which is derived from information theory:

By the Gaussian maximum-entropy theorem (Theorem 8.6.5 in [13]), a continuous random vector $O \in \mathbb{R}^k$ with positive-definite covariance matrix K satisfies the following:

$$h(O) \leq \frac{1}{2} \log((2\pi e)^k \det(K)),$$

with equality iff $O \sim \mathcal{N}(\mu, K)$ for some $\mu \in \mathbb{R}^k$.

Using this entropy bound, we minimize a tractable upper bound on information leakage by minimizing the log-determinant of the output covariance. This leads to the following optimization problem for the linear aggregation layer:

$$\min_W \log \det(K_O) = \log \det(W^T \Sigma W + \sigma^2 I_k). \quad (3)$$

Directly optimizing Equation (3) presents a challenge: since differential entropy is not scale-invariant, one can reduce the determinant of the covariance matrix K by scaling down the weights in W . Without further regularization, the optimizer will reach a trivial solution by pushing W toward the zero matrix.

To address this issue, we need to impose certain constraints on the parameter space of projection matrix W . A natural solution is to let W be an orthonormal matrix, such that $W^T W = I_k$. This constraint ensures that the k projection directions are orthogonal and prevents the objective from collapsing through scaling.

$$\min_{W^T W = I} \log \det(K_O) = \log \det(W^T \Sigma W + \sigma^2 I_k) \quad (4)$$

6.3 Trace-Determinant Equivalence

We show that for fixed Σ , k and $\sigma > 0$, under the orthonormal constraint alone, the log determinant objective has the same optimal subspace as a trace minimization problem, which is well-understood in the context of dimensionality reduction.

Proposition 2: For fixed Σ , k and $\sigma > 0$, with only $W^T W = I$ imposed, the determinant objective in equation (4) can be minimized by the same solution that minimizes the trace objective $\min_{W^T W = I} \text{Tr}(K_O)$. The solution selects the subspace spanned by the k eigenvectors of Σ corresponding to the smallest eigenvalues. $\square$

Proof sketch: Let $v_1 \leq \dots \leq v_n$ be the eigenvalues of Σ , and $\mu_1 \leq \dots \leq \mu_k$ be those of $W^T \Sigma W$. By Poincaré separation [39], for any orthonormal W , $v_i \leq \mu_i \leq v_{n-k+i}$ for $i \in [k]$. Thus both

$$\text{Tr}(K_O) = \sum_{i=1}^k (\mu_i + \sigma^2) \quad \text{and} \quad \log \det(K_O) = \sum_{i=1}^k \log(\mu_i + \sigma^2)$$

are minimized by choosing the smallest possible μ_i 's, namely $\mu_i = v_i$. This is achieved when W spans the eigenvectors of Σ corresponding to its k smallest eigenvalues. Hence the trace and log-determinant objectives have the same minimizer (see [1] for a detailed proof). $\square$

The proposition has two implications.

(1) *Information reduction.* The intuition is that both the log-determinant and trace objectives favor directions with small eigenvalues of Σ . Such directions have low projected variance and therefore lower the information-leakage bound. Selecting them reduces the entropy bound of the aggregated representation, aligning the projection objective with data minimization.

(2) *Relationship to PCA.* Our approach can be viewed as the inverse of principal component analysis (PCA) [27]. PCA selects directions with the largest eigenvalues to preserve maximum variance; in contrast, data minimization favors directions with the smallest eigenvalues to reduce retained information. The objective is to keep only the components needed for the downstream task while suppressing the remaining dimensions for privacy. Masking can create zero-variance directions without task signal; the accuracy constraint restricts which directions are admissible.

Proposition 2 shows that the trace and log-determinant objectives share a minimizer under the sole constraint $W^T W = I$. When the utility constraint is added, we use trace as a differentiable surrogate. For fixed Σ , its gradient $\nabla_W \text{Tr}(W^T \Sigma W + \sigma^2 I_k) = 2\Sigma W$ is linear in W and requires no matrix inversion.

By combining this aggregation objective with the utility constraint, we obtain the final optimization problem for the linear aggregation layer, whose objective is given as follows. For the fixed masked representation, denote by $\mathcal{M}_{\theta, W}$ the predictor with model parameters θ and aggregation matrix W ; the objective is then:

$$\min_{W^T W = I} \text{Tr}(W^T \Sigma W) \quad \text{s.t.} \quad \text{acc}(\mathcal{M}_{\theta, W}) \geq \tau$$

Remark. The trace surrogate is related to the Gaussian variational information bottleneck [3]. For a fixed mask, k and $\sigma > 0$, optimizing an isotropic Gaussian reference distribution gives the leakage bound $\frac{k}{2} \log(1 + \text{Tr}(W^T \Sigma W)/(k\sigma^2))$ (see [1]). Minimizing trace therefore minimizes this bound. M2M couples it with entropy-weighted input removal and accuracy-guided optimization.

6.4 Connection to Reconstruction Error

We next relate trace minimization to expected squared back-projection error. Let $z \in \mathbb{R}^n$ be a centered masked feature vector and $W \in \mathbb{R}^{n \times k}$ an orthonormal projection matrix. Given the compressed representation $o = W^T z + \epsilon$, we map it back to the masked feature space by the Moore-Penrose back-projection $\hat{z} = Wo = W(W^T z + \epsilon)$, and measure reconstruction error by $\|z - \hat{z}\|_2$. For the centered masked data matrix $Z \in \mathbb{R}^{N \times n}$, with one sample per row, the reconstruction is $\hat{Z} = ZWW^T + \mathcal{E}W^T$ where $\mathcal{E}_{ij} \sim \mathcal{N}(0, \sigma^2)$, and the total reconstruction error is:

$$\|Z - \hat{Z}\|_F. \quad (5)$$

We show that, for fixed masked data under an orthonormal projection, trace minimization maximizes the expected squared back-projection error. These conditions match the aggregation stage of M2M: the mask is fixed when optimizing the projection, and orthonormality is already imposed to prevent trivial scaling of W .

Theorem 3: For fixed centered masked data Z , k and σ , and an orthonormal projection matrix W , minimizing the trace of the projected covariance is mathematically equivalent to maximizing the expected squared Frobenius error of the back-projection $\hat{Z}$, that is,

$$\min_{W^T W = I} \text{Tr}(W^T \Sigma W) \iff \max_{W^T W = I} \mathbb{E}_{\mathcal{E}} \|Z - \hat{Z}\|_F^2. \quad \square$$

Proof sketch: Let $P = WW^T$. Since $W^T W = I$, P is an orthogonal projector. With $Z - \hat{Z} = Z(I - P) - \mathcal{E}W^T$, we have:

$$\langle Z(I - P), \mathcal{E}W^T \rangle = \text{Tr}(W^T (I - P) Z^T \mathcal{E}) = 0.$$

Thus $\|Z - \hat{Z}\|_F^2 = \|Z(I - P)\|_F^2 + \|\mathcal{E}W^T\|_F^2$. Moreover, for $\Sigma = \frac{1}{N} Z^T Z$,

$$\|Z(I - P)\|_F^2 / N = \text{Tr}(\Sigma) - \text{Tr}(W^T \Sigma W) \quad \mathbb{E}_{\mathcal{E}} \|\mathcal{E}W^T\|_F^2 = Nk\sigma^2$$

Since $\text{Tr}(\Sigma)$ and $k\sigma^2$ are fixed, minimizing $\text{Tr}(W^T \Sigma W)$ is equivalent to maximizing $\mathbb{E}_{\mathcal{E}} \|Z - \hat{Z}\|_F^2$ (see [1] for a detailed proof). $\square$

The following example illustrates this equivalence.

Example 7: Consider centered data $Z \in \mathbb{R}^{4 \times 3}$ with covariance eigenvalues $v_1 = 2$, $v_2 = 6$ and $v_3 = 12$, and eigenvectors v_1, v_2, v_3 . Let $k = 2$ and $\sigma = 0.1$. Traditional PCA selects the largest-variance directions, $W_{\text{PCA}} = [v_3, v_2]$, giving expected squared back-projection error $4(20 - 18 + 0.02) = 8.08$. Without a utility constraint, trace minimization instead selects $W_{\text{M2M}} = [v_1, v_2]$, yielding $4(20 - 8 + 0.02) = 48.08$. Thus, as Theorem 3 shows, selecting lower-variance directions increases expected squared back-projection error. The data, covariance and sample reconstructions are in [1]. $\square$

Summary. The aggregation stage turns privacy-aware compression into a tractable objective for linear aggregation. From noisy linear aggregation, we derive an entropy-based leakage bound, motivate a trace surrogate under orthonormal projections, and show that it maximizes expected squared back-projection error for fixed masked data. This yields an interpretable, trainable aggregation layer that compresses retained attributes into a less revealing representation.

7 ALGORITHM AND PROPERTIES

This section completes M2M by unifying feature removal and aggregation into a trainable objective. We first formulate the objective and prove its privacy-utility guarantees (Section 7.1), then develop an optimization algorithm with complexity analysis (Section 7.2).

7.1 Unified Objective and Guarantees

We now combine the feature-removal objective from Section 5 with the trace-based aggregation objective from Section 6. Intuitively, M2M performs the following sequence:

$$X \xrightarrow[\text{sparse gate}]{\text{remove}} Z \xrightarrow[\text{trace minimization}]{\text{aggregate}} O \xrightarrow{\text{train } M} Y.$$

That is, raw features X are first masked into Z , and are then compressed into an aggregated representation O , on which the downstream model M is trained. This yields a single constrained objective that jointly learns the mask, aggregation matrix, and model parameters while enforcing the utility requirement.

For a hard-concrete gate g generated from s , let Σ_g be the centered empirical covariance of $g \odot X$ and $p_i(s_i) = \Pr(g_i > 0)$ the activation probability of gate g_i . For utility evaluation, $\mathcal{M}_{\theta, s, W}$ applies the deterministic gate $\hat{g}(s)$, followed by aggregation with W and prediction by the model with parameters θ . Using the marginal-entropy weights w_i and a sparsity coefficient η , M2M seeks a solution to

$$\min_{s, \theta, W^T W = I} \mathbb{E}_g [\text{Tr}(W^T \Sigma_g W)] + \eta \sum_{i=1}^n w_i p_i(s_i) \quad \text{s.t. } \text{acc}(\mathcal{M}_{\theta, s, W}) \geq \tau. \quad (6)$$

Objective (6) makes the privacy-utility tradeoff explicit: the trace and gating terms control the information retained after aggregation and removal, while the constraint enforces downstream utility.

We next show that every feasible solution to this objective admits formal guarantees. Specifically, Theorem 4 gives an upper bound on utility loss and a lower bound on privacy gain for M2M. We evaluate the privacy bound using the deterministic mask, projection and noise of the release, whose covariance can differ from the stochastic training covariance in (6). For a feasible solution (s^*, W^*, θ^*) , we define two quantities used in the bound:

- *Masked covariance:* let $(v_j, u_j)_{j=1}^n$ be the eigenpairs of the centered empirical covariance Σ_Z of $\hat{g}(s^*) \odot X$, where $v_1 \leq \dots \leq v_n$ and the eigenvectors are orthonormal.
- *Subspace alignment:* for the learned aggregation matrix $W^* \in \mathbb{R}^{n \times k}$, its alignment with direction u_j is $P_j(W^*) = \|(W^*)^T u_j\|^2$.

Theorem 4: Fix a feasible (s^*, W^*, θ^*) for (6). For empirical X , let $O = (W^*)^T (\hat{g}(s^*) \odot X) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2 I_k)$ is independent of X and $\sigma > 0$. Then:

$$H(X | O) \geq H(X) - \frac{k}{2} \log \left(1 + \frac{1}{k\sigma^2} \sum_{j=1}^n v_j P_j(W^*) \right), \quad \square$$

$$\text{acc}(\mathcal{M}_{\theta^*, s^*, W^*}) \geq \tau.$$

Proof sketch: The utility bound follows directly from the constraint in Objective (6). For privacy, we lower-bound $H(X | O)$ by upper-bounding the leakage $I(X; O)$. Using the Gaussian maximum-entropy theorem in Section 6.2, we first upper-bound $I(X; O)$ by the log-determinant of the noisy projected covariance. We then use the concavity of the logarithm to further bound this log-determinant by a logarithmic function of the projected trace. Finally, the spectral decomposition of the masked covariance Σ_Z rewrites this trace as a weighted sum of eigenvalues v_j and subspace alignments $P_j(W^*)$, which establishes the bounds (see [1] for details). $\square$

Retained labels. Let B_{tr} denote the logarithmic leakage bound in Theorem 4. With the transformation fixed and release noise independent of (X, Y) , the feature channel satisfies $Y \rightarrow X \rightarrow O$. Thus $I(X; O | Y) = I(X; O) - I(Y; O) \leq B_{\text{tr}}$, giving

$$H(X | O, Y) \geq \max\{0, H(X | Y) - B_{\text{tr}}\}.$$

This bounds uncertainty about the full input vector of a labeled record under the empirical joint distribution. Both $H(X)$ and $H(X | Y)$ are fixed by the input data, so the same trace objective controls the two bounds. The derivation and computed bounds are in [1].

Remarks. (1) The bound depends on the products of two quantities: the variance v_j remaining after feature removal and its alignment $P_j(W^*)$ with the aggregation subspace. Feature removal suppresses task-irrelevant or privacy-sensitive features and changes the masked covariance, while trace-based aggregation favors low-variance directions subject to the accuracy constraint.

(2) The privacy bound in Theorem 4 controls information leakage from X to O , and is relevant beyond reconstruction attacks. For attribute inference, an adversary infers a sensitive attribute X_i from O ; for linkage, it matches O to an auxiliary record. Both rely on information shared between O and the raw features X . The theorem assumes a fixed transformation with public parameters, which matches deployment: M2M first learns the transformation from training data and then applies the fixed transformation to subsequent records. The bound therefore characterizes leakage from released minimized data in this operational phase; it does not cover information disclosed during transformation learning.

7.2 Accuracy-guided Dual Adjustment

We next develop a heuristic solver for Objective (6). Solving this objective raises two challenges: (1) handling the nondifferentiable accuracy constraint; and (2) preserving the orthonormality of the aggregation matrix W . We address the former by applying accuracy-guided dual adjustment (AGDA) to a Lagrangian relaxation, and the latter through QR decomposition after each primal update.

Lagrangian formulation. Denote by $f(s, W) := \mathbb{E}_g[\text{Tr}(W^T \Sigma_g W)] + \eta \sum_{i=1}^n w_i p_i(s_i)$ the data-minimization objective. Relaxing the utility constraint $\text{acc}(\mathcal{M}_{\theta, s, W}) \geq \tau$ yields

$$\min_{s, \theta, W} \max_{\lambda \geq 0} \left[\mathcal{L}(s, \theta, W, \lambda) := f(s, W) + \lambda(\tau - \text{acc}(\mathcal{M}_{\theta, s, W})) \right]. \quad (7)$$

Standard primal-dual solvers, e.g., gradient descent-ascent and stochastic extragradient methods [23], are inapplicable because the indicator-based acc provides no gradient for (s, θ, W) . Replacing accuracy with a differentiable training loss and tuning λ is possible, but evaluating $\mathcal{J}$ multipliers requires $\mathcal{J}$ independent training runs.

Accuracy-guided dual adjustment. AGDA combines a differentiable training loss for primal updates with validation accuracy for grid-based dual adjustment, and thus avoids separate training runs. Denote by $\Lambda = (\lambda_1, \dots, \lambda_{\mathcal{J}})$ a predefined multiplier grid, where $0 \leq \lambda_1 < \dots < \lambda_{\mathcal{J}}$, and by $L_{\mathcal{D}_{\text{train}}}(s, \theta, W)$ a differentiable training loss, e.g., cross-entropy. We obtain $\tilde{f}(s, W)$ by augmenting f with normalized trace $\mathbb{E}_g[\text{Tr}(W^T \Sigma_g W) / \text{Tr}(\Sigma_g)]$ for stability and binary gate regularization $4n^{-1} \sum_i p_i(1 - p_i)$ to encourage binary gates. For a fixed $\lambda \in \Lambda$, AGDA minimizes

$$\tilde{\mathcal{L}}(s, \theta, W; \lambda) = \tilde{f}(s, W) + \lambda L_{\mathcal{D}_{\text{train}}}(s, \theta, W).$$

After each group of primal updates, AGDA evaluates the validation accuracy a and updates the current grid index j as

$$\text{AdjustIndex}(j, a, \tau, \delta, \mathcal{J}) = \begin{cases} \min\{j+1, \mathcal{J}\}, & a < \tau, \\ j, & \tau \leq a \leq \tau + \delta, \\ \max\{j-1, 1\}, & a > \tau + \delta. \end{cases}$$

Input: Schema $\mathcal{R}$, dataset $\mathcal{D} = (\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{val}}, \mathcal{D}_{\text{test}})$ of $\mathcal{R}$, model $\mathcal{M}$, multiplier grid $\Lambda = (\lambda_1, \dots, \lambda_{\mathcal{J}})$, optimizer OPT, T, q, δ , and τ .

Output: $s, \tilde{g}(s), \theta$, and W .

1. Initialize s_0^0, θ_0^0 randomly, $j^0 := \mathcal{J}$, and $\lambda^0 := \lambda_{\mathcal{J}}$;
2. Initialize W_0^0 such that $(W_0^0)^T W_0^0 = I$;
3. **for** $t = 0$ to $T - 1$ **do**
4. **for** $i = 0$ to $q - 1$ **do**
5. Sample a mini-batch $B_i^t \subset \mathcal{D}_{\text{train}}$
6. $(s_{i+1}^t, \theta_{i+1}^t, \tilde{W}_{i+1}^t) := \text{OPT}(\tilde{\mathcal{L}}_{B_i^t}(\cdot; \lambda^t), s_i^t, \theta_i^t, W_i^t)$;
 // QR decomposition to preserve orthonormality
7. $W_{i+1}^t := \text{QR}(\tilde{W}_{i+1}^t)$;
8. $(s_0^{t+1}, \theta_0^{t+1}, W_0^{t+1}) := (s_q^t, \theta_q^t, W_q^t)$;
 // Accuracy-guided dual adjustment
9. $a^{t+1} := \text{acc}_{\mathcal{D}_{\text{val}}}(\mathcal{M}_{\theta_0^{t+1}, s_0^{t+1}, W_0^{t+1}})$;
10. $j^{t+1} := \text{AdjustIndex}(j^t, a^{t+1}, \tau, \delta, \mathcal{J})$;
11. $\lambda^{t+1} := \lambda_{j^{t+1}}$;
12. **if** $a^t < \tau$ for all $t \in [1, T]$ **then return** failure;
13. $t^* := \arg \min_{1 \leq t \leq T, a^t \geq \tau} f(s_0^t, W_0^t)$;
14. **return** $(s_0^{t^*}, \tilde{g}(s_0^{t^*}), \theta_0^{t^*}, W_0^{t^*})$;

Figure 2: Algorithm M2M_AGDA for data minimization

Algorithm. Putting these together, Figure 2 presents M2M_AGDA. It takes as input $\mathcal{R}, \mathcal{D}, \mathcal{M}, \Lambda$, a primal optimizer OPT, τ, δ , an outer-iteration budget T , and an adjustment interval q . At each outer iteration, it performs q primal updates at a fixed multiplier before adjusting the multiplier using validation accuracy. This double-loop structure allows the primal variables to respond to the multiplier before its next adjustment. In each inner iteration, M2M_AGDA samples a mini-batch from $\mathcal{D}_{\text{train}}$ (line 5), and updates s, θ and W with OPT using a reparameterized hard-concrete gate (line 6). It restores the orthonormality of W by applying QR decomposition to $\tilde{W}$ (line 7). The updates to s, W and θ respectively learn feature-removal gates under the entropy-weighted sparsity penalty, minimize the projected covariance trace, and fit the downstream model. After q updates, it carries the primal variables to the next outer iteration (line 8), computes validation accuracy (line 9), and updates the grid index and multiplier (lines 10–11). It reports when no feasible iterate is found (line 12); otherwise, it returns the validation-feasible iterate with the smallest data-minimization objective (lines 13–14).

Example 8: Suppose that $\Lambda = \{0, 0.1, 0.3, 0.5, 0.7, 0.9\}$, the current multiplier is $\lambda = 0.5$, $\tau = 0.80$, and $\delta = 0.05$. During the inner loop, M2M_AGDA keeps $\lambda = 0.5$ fixed and updates s, θ and W for q steps by minimizing $\tilde{\mathcal{L}}$. It then computes validation accuracy a and updates the multiplier as follows: (1) it increases λ to 0.7 if $a = 0.72 < \tau$; (2) it keeps $\lambda = 0.5$ if $a = 0.83 \in [\tau, \tau + \delta]$; and (3) it decreases λ to 0.3 if $a = 0.88 > \tau + \delta$. $\square$

Complexity. Let $b = |B_i^t|$ be the mini-batch size, n be the number of gates, and k be the aggregation dimension. One primal update costs $\text{cost}_{\text{OPT}}(b)$, while one validation pass costs $c_{\text{fwd}} = \text{cost}_{\text{fwd}}(N_{\text{val}})$. Including the $O(nk^2)$ QR decomposition, each outer iteration takes $O(q(\text{cost}_{\text{OPT}}(b) + nk^2) + c_{\text{fwd}})$ time, with $O(1)$ index adjustment. For a first-order optimizer and a model $\mathcal{M}$ with p trainable parameters, $\text{cost}_{\text{OPT}}(b) = O(b(p + nk))$. Taken together, T outer iterations take $O(T(qb(p + nk) + qnk^2 + c_{\text{fwd}}))$ time in total.

Remark. (1) δ does not relax τ : AGDA increases λ if $a < \tau$ and keeps it unchanged only in $[\tau, \tau + \delta]$. The interval reduces oscillation

between adjacent grid points for feasible accuracies near τ . If no feasible iterate is found, AGDA reports failure for the grid and budget. (2) The test split $\mathcal{D}_{\text{test}}$ is used only for final evaluation, not primal optimization, multiplier adjustment, or feasible-iterate selection.

8 EXPERIMENTAL STUDY

Using real-life and synthetic datasets, we evaluated the effectiveness and efficiency of M2M for utility-constrained data minimization.

Experimental setting. We start with our settings.

Datasets. We evaluated M2M on five real-life tabular datasets: CoverType, Adult, Mushroom, NATICUSdroid and Marketing. Table 1 summarizes their statistics and prediction tasks. We split each dataset 60/20/20 into training, validation, and test sets. For baselines without continuous-feature support, numerical features were discretized and all features were one-hot encoded, with all preprocessing parameters fit on the training split only and then applied to validation and test data to avoid data leakage. Each encoded column has an independent gate; an original attribute’s gate value and entropy weight are averaged over its columns. An original attribute is fully removed only when all its column gates are zero. We also generated synthetic data with $n = 200$ attributes by default: 100 high-entropy and 100 low-entropy attributes, following balanced and skewed binary distributions, respectively. Labels depend on all attributes, whose values are variance-standardized to give the two groups comparable contributions (see [1] for details).

Baselines. We implemented M2M in Python and compared it with four baselines for vertical feature minimization or transformation: (1) PAT [56], a vertical data minimization method based on tree-based generalization; (2) MetaModeling [19], a personalized data minimization method that we adapt to group-level minimization by sharing one strategy across all examples; (3) ANOVA, a filter-based feature selection baseline that ranks features by association with the label; and (4) Representative [22], a model-aware generalization method that learns and prunes a decision-tree generalizer under an accuracy threshold and maps each leaf to a representative input value. We exclude horizontal minimization baselines, which remove tuples rather than per-tuple features and are therefore not comparable under the same reconstruction metric.

Classifiers $\mathcal{M}$. We tested three downstream classifiers for utility evaluation: (1) logistic regression [49], (2) multilayer perceptron (MLP) [36], and (3) FT-Transformer (FTT) [24]. By default, $\mathcal{M}$ is an MLP with two 256-unit hidden layers.

Utility constraint. On the validation split, we compute two reference accuracies: (1) majority accuracy (MA) using the majority-class predictor; and (2) full-data accuracy (FDA) using $\mathcal{M}$ trained on the original training split with all input attributes. Given a retention level x , we instantiate the utility threshold τ in Equation (1) as:

$$\tau(x) = \text{MA} + x \cdot (\text{FDA} - \text{MA}).$$

For example, $\tau(90)$ preserves 90% of the accuracy gain from MA to FDA. During configuration selection, a configuration is *utility-feasible* iff its validation accuracy is at least $\tau(x)$; the test set is used only to evaluate the selected configuration.

Metrics. Since direct estimation of the mutual-information objective $I(X; \mathcal{O})$ is unstable on high-dimensional tabular data, we evaluate

Dataset	$ \mathcal{D} $	#Attr.	#Emb.	Task
CoverType [25]	495,141	54	188	Forest cover type prediction
Adult [8]	48,842	14	133	Census income classification
Mushroom [60]	8,124	22	115	Edibility prediction
NATICUSdroid [41]	29,333	86	86	Malware detection
Marketing [36]	8,993	13	83	Household income prediction

Table 1: Datasets (Attr.: attributes, Emb.: encoded features).

privacy using two attack-based metrics: (1) feature reconstruction, which covers attribute inference, and (2) candidate-record linkage, which measures re-identification among the supplied candidates. For feature reconstruction, we train an adversary $\mathcal{A}$ on auxiliary data $\mathcal{D}'$ sampled from the training split to infer the raw feature vector x from its minimized representation x_M . Let $r(x)$ be the vector of reconstruction targets, with one entry per original input attribute and d_{rec} entries in total. Categorical decoding is specified in [1]. We test (a) logistic regression, (b) multilayer perceptron (MLP), and (c) FT-Transformer (FTT), using MLP by default. The average feature-wise reconstruction accuracy is

$$\text{racc} := \mathbb{E}_{(x, x_M)} \left[\frac{1}{d_{\text{rec}}} \sum_{i=1}^{d_{\text{rec}}} \mathbb{1}(\mathcal{A}(x_M)_i = r(x)_i) \right].$$

Because racc can be high on imbalanced features even under majority guessing, we normalize it relative to the majority reconstruction accuracy MRA, obtained by predicting each feature’s majority value, and define the *normalized reconstruction risk* nrr as

$$\text{nrr} := (\text{racc} - \text{MRA}) / (1 - \text{MRA}).$$

Thus, lower nrr indicates less reconstruction gain over majority guessing and hence lower reconstruction risk. Cross-dataset summaries weight the dataset means equally. Relative reductions are computed against the comparator on each dataset and then averaged; they are not ratios of the averaged risks. Unless stated otherwise, we compare methods by nrr at the same utility threshold τ ; for utility-privacy tradeoffs, we also report downstream accuracy acc.

For linkage attacks, a black-box attacker receives auxiliary released-row pairs, but no transformation parameters or query interface. Following [46], we use a pairwise classifier to score released-row record pairs and link each released record to the highest-scoring raw record among 200 candidates. We report *attack success rate* (asr), the fraction of released records linked to their true matches; lower asr indicates lower linkage risk.

Configurable parameters. We use utility threshold $\tau(50)$ for Adult and $\tau(80)$ for other datasets, select the aggregation dimension k from $\{2, 8\}$ and the Gaussian-noise scale σ from $\{0, 0.05, 0.25\}$, and set the sparsity coefficient to $\eta = 0.2$. For M2M-AGDA, we use multiplier grid $\Lambda = \{0, 0.1, 0.2, \dots, 1, 1.2, 1.5\}$, accuracy margin $\delta = 0.01$, adjustment interval q of one epoch, and primal optimizer OPT = Adam [29]. Unless stated otherwise, all experiments use these settings. We set a timeout of 2h for each run. Selected settings and utility summaries are reported in [1].

For baselines, we follow the settings recommended in their original papers or released implementations and sweep only the parameters controlling minimization. We retain configurations satisfying τ on the validation split and select one according to each method’s privacy criterion when available; otherwise, we choose its strongest privacy setting: the largest α for PAT and the fewest selected features for ANOVA and MetaModeling. The selected configurations are evaluated using the same attacks and metrics as M2M.

Datasets	Remove	Retain	k	Comp. ratio	Trace red.	acc	nrr
CoverType	101.67	86.33	8	4.26%	70.51%	83.52%	48.17%
Adult	52.67	80.33	2	1.50%	60.96%	81.82%	11.66%
Mushroom	23.33	91.67	2	1.74%	85.94%	91.30%	7.46%
NATICUSdroid	42	44	2	2.33%	71.22%	89.49%	18.19%
Marketing	20.67	62.33	2	2.41%	41.39%	76.47%	18.20%

Table 2: M2M statistics (counts of encoded columns).

Configuration. We ran experiments on a machine with 504 GB RAM, Intel Xeon Gold 5320 processors at 2.20 GHz, and 4 NVIDIA RTX 3090 24 GB GPUs. We repeated each experiment 3 times and report the average. Repetition protocols and standard deviations for main comparisons and component ablations are reported in [1].

Experimental results. We report results on representative datasets; the other datasets show consistent trends and are omitted.

Exp-1: Effectiveness. We first evaluated whether M2M reduces recoverable raw-feature information under a fixed utility constraint.

Normalized reconstruction risk vs. baselines. As Figure 3(a) shows, M2M reduces nrr relative to PAT, the strongest baseline on all five datasets, by 32.25% on average and up to 44.46%; mean nrr drops from 30.36% to 20.74%. The lower accuracy reflects the intended privacy-utility tradeoff: M2M minimizes retained information subject to the prescribed accuracy threshold, rather than maximizing accuracy beyond it. For Adult, nrr drops by 36.32% with only a 0.0001 accuracy decrease; for Mushroom, the reductions are 21.06% and 0.0720, respectively. Per-dataset accuracy and variability are reported in [1]. In contrast, Representative finds no validation-feasible CoverType result within the time limit.

Minimization-quality statistics. On average, M2M removes 48.07 encoded columns and releases 3.20 aggregates. The mean ratio $k/\#Emb$ is 2.45%; projected trace falls by 66.00% on average (see Table 2). It fully removes 16.64% of original attributes on average; removed, partial and retained counts are in [1]. With preprocessing at the source, fully removed attributes need not be acquired.

Privacy-utility Pareto frontier. On Adult, for each method, we generate candidate minimized representations and keep those satisfying the utility threshold $\tau(50)$. Each point in Figure 3(c) is one feasible configuration, with acc as utility and nrr as privacy risk. As shown in Figure 3(c), M2M dominates the baselines: relative to the best baseline, at the same acc, it reduces nrr by 41.86% on average, up to 52.58%; at the same nrr, it improves acc by 1.68% on average.

Feature-level interpretability. Table 3 reports six representative Adult features. M2M fully removes attributes fnlwtg, sex and hours. It partially retains task-relevant attributes such as age, marital and occupation. Thus fully removed attributes need not be acquired. Partial retention still requires the source value: the retained age indicator, e.g., must be computed from the original age.

Trace and reconstruction. Figure 3(d) compares utility-feasible configurations on Mushroom, with nrr ranging from 0.0557 to 0.2039 and accuracy from 0.848 to 0.979. Smaller projected trace is associated with lower nrr and larger reported reconstruction error. A larger trace preserves more projected variance, potentially retaining more task-relevant signal but exposing more reconstructable information. These results support M2M’s trace surrogate; settings, trace normalization and error calculation are in [1].

Attack-level privacy. We tested M2M against linkage attacks. As

Metric	age	marital	occupation	fnlwtg	sex	hours
Mean gate	0.20	0.57	0.33	0.00	0.00	0.00
w_j	0.325	0.296	0.226	0.325	0.634	0.366
Kept signal	age < 26	married/never	selected jobs	none	none	none
Retain?	Partial	Partial	Partial	No	No	No

Table 3: Feature-level case study of M2M on Adult.

shown in Figure 3(e), M2M reduces asr to 4.05%, versus 8.88% for the best baseline, improving by 33.52% on average and up to 63.51%.

Leakage-bound tightness. We tested the bound on Gaussian reference channels with $\sigma > 0$. Using Gaussian synthetic data, we varied k , σ , η and τ to get feasible M2M solutions. For each solution, we construct a Gaussian reference input X with the same covariance as the training data and apply the fixed learned mask and projection. Denote by Σ_Z the resulting masked covariance. The exact leakage is $I(X; O) = I(Z; O) = \frac{1}{2} \log \det(I + \sigma^{-2} W^\top \Sigma_Z W)$, compared with $B_{th} = \frac{k}{2} \log(1 + \sum_j v_j P_j(W)/(k\sigma^2))$. As Figure 3(f) shows, the leakage is below B_{th} for every channel, with an average gap of 19.46%.

For fixed real-data releases with retained labels, the empirical residual-entropy lower bound is positive in all 3 runs on all 4 datasets with $\sigma > 0$, with zero test-accuracy shortfall below τ in every selected run. Per-dataset shortfalls and bound ranges are in [1].

Varying reconstruction attackers. We first varied the reconstruction attacker architecture over (1) logistic regression, (2) MLP, and (3) FT-Transformer on Adult. We further considered MLP_{adp} , a noiseless adaptive variant of MLP, the best of these architectures. After each method learns and fixes its transformation ϕ , MLP_{adp} trains and tests on evaluator-supplied noiseless representations. It trains a separate MLP for each method on the resulting representation-raw pairs. The same raw $\mathcal{D}'$ is used across methods, and ϕ remains fixed.

As Figure 3(g) shows, across the three standard attackers, M2M’s nrr ranges from 3.99% to 9.93%. Compared with the lowest-nrr baseline for each attacker, M2M reduces nrr by 49.58% on average, up to 57.00%. Under MLP_{adp} , bypassing Gaussian noise increases M2M’s nrr from 9.93% to 14.34%. This said, M2M remains below every baseline: it reduces nrr by 21.68% relative to the strongest baseline.

Varying $|\mathcal{D}'|$. We varied the adversary’s auxiliary data size $|\mathcal{D}'|$ from 20% to 100% on Marketing. As shown in Figure 3(h), nrr increases from 14.32% to 17.85% as $|\mathcal{D}'|$ grows. Even with the full auxiliary set, M2M remains 45.5% lower than the best baseline.

Varying τ . We varied the accuracy threshold τ by setting the retention level x in $\tau(x)$ to $\{90, 85, 80, 75\}$ on NATICUSdroid. As Figure 3(i) shows, nrr increases by 55.16%, from 17.15% to 26.61%, as the utility threshold tightens, and decreases when more utility loss is allowed. A higher τ leaves less room for feature removal and aggregation, forcing M2M to retain more task-relevant information.

Varying $\mathcal{M}$. On Marketing, we reran minimization for each downstream classifier $\mathcal{M}$ under the same utility threshold. As shown in Figure 3(j), M2M remains utility-feasible for all tested classifiers and reduces nrr relative to the strongest baseline by 55.81% on average, up to 62.27% (with $\mathcal{M} = \text{FT-Transformer}$). Varying $\mathcal{M}$ changes the task loss and utility constraint used to learn the feature gates and aggregation map. The entropy-weighted removal and trace objectives remain the same across classifiers.

Optimizer and adjustment settings. SGD, Adam, AdamW, and RMSprop all find utility-feasible solutions on Mushroom; RMSprop

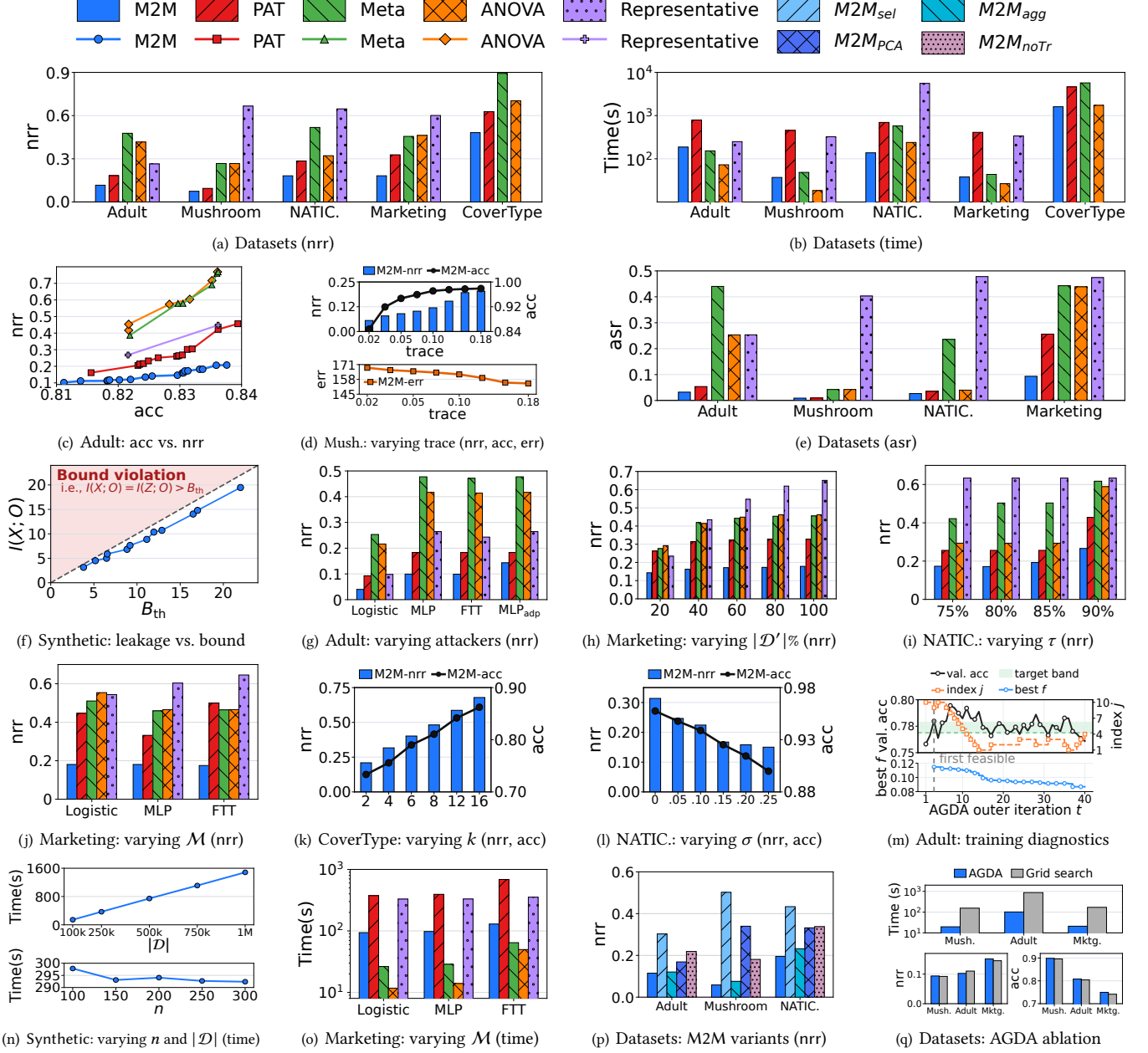


Figure 3: Performance evaluation

reduces nrr by 23.20% relative to SGD. Sweeps over q , δ , Λ , and T also remain feasible. The nrr ranges are 20.32–20.62% for q and 9.49–9.88% for δ ; across Λ , the minimum is only 0.99% below the maximum. These three sweeps show no monotonic trend in nrr or acc. As T increases, nrr generally decreases up to $T = 40$ and then fluctuates, while runtime increases (see [1]).

The k and σ sweeps fix other settings and use configurations different from the main comparison (see [1]).

Impact of k . We varied the aggregation dimension k on CoverType. As shown in Figure 3(k), increasing k from 2 to 16 improves accuracy from 0.733 to 0.862, but also raises nrr from 0.210 to 0.679. A larger k preserves more retained-feature dimensions, improving utility

while exposing more reconstructable information. From $k = 8$ to $k = 16$, accuracy increases by 0.052, while nrr increases by 0.197. Thus, we use $k = 8$ as the tradeoff point within this sweep.

Impact of σ . We varied the Gaussian-noise scale σ from 0 to 0.25 on NATICUSdroid. As shown in Figure 3(l), increasing σ reduces nrr from 0.314 to 0.150, while accuracy drops from 0.958 to 0.900. This is because Gaussian noise smooths the aggregated representation and hinders reconstruction, but excessive noise also perturbs task-relevant signals. The first noise step, $\sigma = 0.05$, reduces nrr by 0.067 and accuracy by 0.010, giving the best marginal tradeoff.

Exp-2: Efficiency and scalability. We compare end-to-end runtime with baselines, and evaluate how M2M scales with data

size, feature dimension, and downstream classifier.

Runtime. Figure 3(b) reports end-to-end runtime under the default configuration. Across datasets, M2M takes 37.05–1603.41s. Compared with PAT, M2M is faster with an average speedup of 7.03 $\times$, up to 12.22 $\times$ on Mushroom. Compared with Representative, M2M is faster with an average speedup of 14.86 $\times$, up to 40.46 $\times$ on NATICUSdroid. ANOVA is faster on Adult, Mushroom and Marketing, while MetaModeling is faster only on Adult. However, these methods incur substantially higher nrr.

Training diagnostics. We traced M2M_AGDA for 40 outer iterations on Adult. Figure 3(m) (up) shows validation accuracy a^t and the target band; the first feasible iterate is $t = 3$, with $a^3 = 0.7798$. The lower panel shows $f_{\text{best}}^t = \min_{1 \leq r \leq t, a^r \geq \tau} f(s_0^r, W_0^r)$, the best objective among feasible iterates. It decreases from 0.115 to 0.087, with the last improvement at iteration 37.

Varying $|\mathcal{D}|$ and n . On the default synthetic data, runtime grows linearly from 144s to 1484s as $|\mathcal{D}|$ increases from 100k to 1M with $n = 200$ (Figure 3(n), up). This is because M2M trains the downstream model and estimates covariance statistics over the data. With $|\mathcal{D}| = 200k$, runtime stays between 292s and 298s when the number n of raw features grows from 100 to 300 (Figure 3(n), down). The small, fixed aggregation dimension limits the impact of increasing n on the number of M2M parameters and overall training cost.

Varying $\mathcal{M}$. We measured runtime using the classifiers and settings of Figure 3(j) on Marketing. As shown in Figure 3(o), M2M takes longer with more complex classifiers, e.g., FT-Transformer, because minimization requires training $\mathcal{M}$. Across classifiers, the runtime varies by 35.73s, with an average runtime of 107.05s.

Exp-3: Ablation. We finally evaluated M2M through four tests: component ablation, synthetic ground-truth validation, mixed-type robustness, and M2M_AGDA ablation.

Component ablation. We compared M2M with four variants under the same utility threshold (see Figure 3(p)). We find the following.

- (1) M2M outperforms M2M_{sel}, which uses entropy-weighted feature removal but releases the selected raw features without aggregation. Its mean nrr is 41.27%, compared with 12.27% for M2M, because selection alone still exposes retained raw attributes, whereas M2M further compresses them via trace-minimizing aggregation.
- (2) M2M_{agg} retains all inputs and applies trace-minimizing aggregation. Within the same linear representation family, M2M removes 43.54% of original attributes and reduces nrr by 14.58% on average, with mean nrr falling from 14.28% to 12.27%. Mean test accuracy decreases by 0.0349 relative to M2M_{agg}. Per-dataset results and variability are reported in [1].
- (3) M2M outperforms M2M_{PCA}, which keeps the two-stage pipeline but replaces trace minimization with PCA over the retained features. Its mean nrr is 28.02%, compared with 12.27% for M2M, because PCA preserves high-variance directions, whereas trace minimization reduces projected variance subject to the accuracy constraint.
- (4) M2M_{noTr} omits the trace term from both training and checkpoint selection, retaining trainable gates, projection and classifier with matched k , noise and training budgets. Compared with M2M_{noTr}, full M2M reduces nrr by 52.49% on average, up to 67.70%, with mean

nrr falling from 24.57% to 12.27%. Mean test accuracy decreases by 0.0341; per-dataset results and variability are in [1]. These results support gating and the trace objective in this linear family.

Synthetic ground-truth validation. On synthetic data with all features task-relevant, we compared M2M with M2M_{unif}, which weights all features equally. M2M removes 98.67% of high-entropy features and no low-entropy features on average, versus 65.00% and 72.33% for M2M_{unif}. Their nrr values are 10.12% and 34.21%, respectively, under the same utility target. Thus entropy weighting concentrates removal on high-entropy features (see [1] for details).

Mixed-type robustness. We used a mixed synthetic dataset with 100 categorical and 100 continuous features, evenly split by type within each feature-role group. We compared M2M with all baselines except PAT, whose PGini criterion assumes finite-valued personal attributes and cannot directly protect continuous attributes. For continuous features X_c and reconstructions $\hat{X}_c$, we report $\text{nrmse} = \|X_c - \hat{X}_c\|_F / \|X_c\|_F$, where a larger value indicates lower recoverability. Among utility-feasible configurations, the average nrmse of M2M is 0.99, 39.03% higher than the best baseline. This shows that M2M reduces recoverability for raw continuous features (see [1]).

Ablation of M2M_AGDA. We tested M2M_AGDA with grid search over the same multiplier grid Λ using the same primal optimizer and per-run training budget. Grid search performs one independent optimization run for each $\lambda \in \Lambda$. For both methods, we select the validation-feasible solution with the smallest data-minimization objective and use the test set only for final evaluation. As shown in Figure 3(q), on average, M2M_AGDA takes 47.5s, compared with 399.6s for grid search, a speedup of 8.41 $\times$. It achieves test acc = 81.87% and nrr = 11.6%, compared with acc = 81.39% and nrr = 11.6% for grid search. Thus, M2M_AGDA reaches comparable privacy and utility using one training run instead of $|\Lambda|$ independent runs (the comparison grid and budgets are given in [1]).

Summary. (1) *Effectiveness.* Under the same utility constraint, M2M reduces nrr by 32.25% on average over the best baseline, up to 44.46%. (2) *Robustness.* The advantage persists across reconstruction attackers, auxiliary-data sizes, and classifiers; linkage attacks show the same trend. (3) *Parameters.* Larger k improves utility but increases nrr, while larger σ has the opposite effect. (4) *Efficiency.* Runtime scales from 144s at 100K examples to 1484s at 1M examples, and stays within 292–298s as the feature dimension grows from 100 to 300. (5) *Ablation.* Gating reduces mean nrr from 14.28% to 12.27%, while the trace objective reduces it from 24.57% to 12.27%. (6) *Recommended setting.* We use $k = 2$, $\sigma = 0.05$, and $T = 200$ by default.

9 CONCLUSION

M2M combines entropy-weighted input removal and trace-based aggregation under an accuracy constraint. An accuracy-guided algorithm learns reusable transformations. For fixed positive-noise releases, we bound leakage under the empirical input distribution. Experiments show lower reconstruction risk against feature-only learned attackers than the tested baselines; component ablations support gating and the trace objective within the same linear family.

Future research directions include (a) extending the framework beyond classification models and relational data, and (b) jointly balancing accuracy, fairness and robustness of downstream ML models.

REFERENCES

- [1] 2026. Code, datasets and full version. <https://github.com/xyzhou055/m2m-paper>.
- [2] Martin Abadi, Andy Chu, Ian Goodfellow, H. Brendan McMahan, Ilya Mironov, Kunal Talwar, and Li Zhang. 2016. Deep Learning with Differential Privacy. In *ACM Conference on Computer and Communications Security (CCS)*. 308–318.
- [3] Alexander A Alemi, Ian Fischer, Joshua V Dillon, and Kevin Murphy. 2017. Deep Variational Information Bottleneck. In *International Conference on Learning Representations*.
- [4] Rana Ali Amjad and Bernhard C Geiger. 2020. Learning representations for neural network-based classification using the information bottleneck principle. *IEEE transactions on pattern analysis and machine intelligence* 42, 9 (2020), 2225–2239. <https://doi.org/10.1109/TPAMI.2019.2909031>
- [5] Meenatchi Sundaram Muthu Selva Annamalai, Georgi Ganev, and Emiliano De Cristofaro. 2024. "What Do You Want from Theory Alone?" Experimenting with Tight Auditing of Differentially Private Synthetic Data Generation. In *USENIX Security Symposium (Security)*. 4855–4871. <https://www.usenix.org/conference/usenixsecurity24/presentation/annamalai-theory>
- [6] Raef Bassily, Adam Smith, and Abhradeep Thakurta. 2014. Private empirical risk minimization: Efficient algorithms and tight error bounds. In *IEEE annual symposium on foundations of computer science*. IEEE, 464–473.
- [7] Roberto Battiti. 1994. Using mutual information for selecting features in supervised neural net learning. *IEEE Transactions on neural networks* 5, 4 (1994), 537–550.
- [8] Barry Becker and Ronny Kohavi. 1996. Adult. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5XW20>.
- [9] Asia J. Biega, Peter Potash, Hal Daumé, Fernando Diaz, and Michèle Finck. 2020. Operationalizing the Legal Principle of Data Minimization for Personalization. In *ACM Conference on Research and Development in Information Retrieval (SIGIR)*. 399–408. <https://dl.acm.org/doi/10.1145/3397271.3401034>
- [10] Zalán Borsos, Mojmir Mutny, and Andreas Krause. 2020. Coresets via Bilevel Optimization for Continual Learning and Streaming. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [11] Léon Bottou. 2010. Large-scale machine learning with stochastic gradient descent. In *International Conference on Computational Statistics (COMPSTAT)*. Springer, 177–186.
- [12] Liam Collins, Hamed Hassani, Aryan Mokhtari, and Sanjay Shakkottai. 2021. Exploiting shared representations for personalized federated learning. In *International conference on machine learning*. PMLR, 2089–2099.
- [13] Thomas M. Cover and Joy A. Thomas. 2006. *Elements of Information Theory* (2nd ed.). John Wiley & Sons, Hoboken, NJ. <https://doi.org/10.1002/047174882X>
- [14] Elvis Dohmatob, Mohammad Pezeshki, and Reyhane Askari Hemmat. 2025. Why Less is More (Sometimes): A Theory of Data Curation. *CoRR* abs/2511.03492 (2025). <https://doi.org/10.48550/ARXIV.2511.03492>
- [15] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography (TCC)*. 265–284.
- [16] Harrison Edwards and Amos Storkey. 2016. Censoring Representations with an Adversary. In *International Conference on Learning Representations*.
- [17] Alireza Fallah, Aryan Mokhtari, and Asuman Ozdaglar. 2020. Personalized federated learning: A meta-learning approach. *arXiv preprint arXiv:2002.07948* (2020).
- [18] Jean Feng and Noah Simon. 2017. Sparse-input neural networks for high-dimensional nonparametric regression and classification. *arXiv preprint arXiv:1711.07592* (2017).
- [19] Prakhar Ganes, Cuong Tran, Reza Shokri, and Ferdinando Fioretto. 2025. The Data Minimization Principle in Machine Learning. In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 3075–3093. <https://doi.org/10.1145/3715275.3732195>
- [20] Michael Garey and David Johnson. 1979. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company.
- [21] Ziv Goldfeld, Ewout van den Berg, Kristjan Greenewald, Igor Melnyk, Nam Nguyen, Brian Kingsbury, and Yuri Polyanskiy. 2018. Estimating information flow in deep neural networks. *arXiv preprint arXiv:1810.05728* (2018).
- [22] Abigail Goldstein, Gilad Ezov, Ron Shmelkin, Micha Moffie, and Ariel Farkash. 2022. Data Minimization for GDPR Compliance in Machine Learning Models. *AI and Ethics* 2, 3 (Aug. 2022), 477–491. <https://link.springer.com/10.1007/s43681-021-00095-8>
- [23] Eduard Gorbunov, Hugo Berard, Gauthier Gidel, and Nicolas Loizou. 2022. Stochastic extragradient: General analysis and improved rates. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 7865–7901.
- [24] Yuri Gorishniy, Ivan Rubachev, Valentin Khurlov, and Artem Babenko. 2021. Revisiting Deep Learning Models for Tabular Data. In *NIPS*. 18932–18943.
- [25] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data? *Advances in neural information processing systems* 35 (2022), 507–520.
- [26] Geoffrey Hinton. 2012. RMSProp: Divide the Gradient by a Running Average of Its Recent Magnitude. COURSE: Neural Networks for Machine Learning.
- [27] Harold Hotelling. 1933. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology* 24, 6 (1933), 417–441. <https://doi.org/10.1037/h0071325>
- [28] Yuzheng Hu, Fan Wu, Qibin Li, Yunhui Long, Gonzalo Munilla Garrido, Chang Ge, Bolin Ding, David Forsyth, Bo Li, and Dawn Song. 2024. SoK: Privacy-Preserving Data Synthesis. In *IEEE Symposium on Security and Privacy (SP)*. 4696–4713. <https://www.computer.org/csdl/proceedings-article/sp/2024/313000a002/1RjE9GWpTGw>
- [29] Diederik P Kingma and Jimmy Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980* (2014).
- [30] Artemy Kolchinsky, Brendan D Tracey, and Steven Van Kuyk. 2018. Caveats for information bottleneck in deterministic scenarios. *arXiv preprint arXiv:1808.07593* (2018).
- [31] Elahe Ghasemi Komishani, Mahdi Abadi, and Fatemeh Deldar. 2016. PPTD: Preserving personalized privacy in trajectory data publishing by sensitive attribute generalization and trajectory local suppression. *Knowledge-Based Systems* 94 (2016), 43–59.
- [32] Sanjay Krishnan, Jiannan Wang, Michael J. Franklin, Ken Goldberg, and Tim Kraska. 2016. PrivateClean: Data Cleaning and Differential Privacy. In *SIGMOD*. 937–951. <https://doi.org/10.1145/2882903.2915248>
- [33] Juyan Li, Zhiqi Qiao, Kejia Zhang, and Chen Cui. 2021. A lattice-based homomorphic proxy re-encryption scheme with strong anti-collusion for cloud computing. *Sensors* 21, 1 (2021), 288.
- [34] Kecen Li, Chen Gong, Zhixiang Li, Yuzhong Zhao, Xinwen Hou, and Tianhao Wang. 2024. PrivImage: Differentially Private Synthetic Image Generation Using Diffusion Models with Semantic-Aware Pretraining. In *USENIX Security Symposium (Security)*. 4837–4854. <https://www.usenix.org/conference/usenixsecurity24/presentation/li-kecen>
- [35] Qiongxiu Li, Ignacio Cascudo, and Mads Græsbøll Christensen. 2019. Privacy-preserving distributed average consensus based on additive secret sharing. In *European Signal Processing Conference (EUSIPCO)*. IEEE, 1–5.
- [36] Zifan Liu, Zhechun Zhou, and Theodoros Rekatsinas. 2020. Picket: Self-supervised Data Diagnostics for ML Pipelines. *CoRR* abs/2006.04730 (2020). <https://arxiv.org/abs/2006.04730>
- [37] Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101* (2017).
- [38] Christos Louizos, Max Welling, and Diederik P Kingma. 2017. Learning sparse neural networks through L_0 regularization. *arXiv preprint arXiv:1712.01312* (2017).
- [39] Jan R Magnus and Heinz Neudecker. 2019. *Matrix differential calculus with applications in statistics and econometrics* (3rd ed.). John Wiley & Sons.
- [40] Ali Makhdoom, Salman Salamati, Nadia Fawaz, and Muriel Médard. 2014. From the Information Bottleneck to the Privacy Funnel. In *IEEE Information Theory Workshop*. IEEE, 501–505.
- [41] Akshay Mathur, Laxmi Mounika Podila, Keyur Kulkarni, Quamar Niyaz, and Ahmad Y Javadi. 2021. NATICUSDroid: A malware detection framework for Android using native and custom permissions. *Journal of Information Security and Applications* 58 (2021), 102696.
- [42] Yoshitomo Matsubara, Ruihan Yang, Marco Levorato, and Stephan Mandt. 2022. Supervised compression for resource-constrained edge computing systems. In *IEEE/CVF winter conference on applications of computer vision*. 2685–2695.
- [43] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, and Blaise Agüera y Arcas. 2017. Communication-efficient learning of deep networks from decentralized data. In *Artificial intelligence and statistics*. PMLR, 1273–1282.
- [44] Baharan Mirzasoleiman, Jeff A. Bilmes, and Jure Leskovec. 2020. Coresets for Data-efficient Training of Machine Learning Models. In *ICML (Proceedings of Machine Learning Research)*, Vol. 119. PMLR, 6950–6960.
- [45] Baharan Mirzasoleiman, Kaidi Cao, and Jure Leskovec. 2020. Coresets for Robust Training of Deep Neural Networks against Noisy Labels. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [46] Sidharth Mudgal, Han Li, Theodoros Rekatsinas, AnHai Doan, Youngchoon Park, Ganesh Krishnan, Rohit Deep, Esteban Arcaute, and Vijay Raghavendra. 2018. Deep Learning for Entity Matching: A Design Space Exploration. In *SIGMOD*. 19–34.
- [47] Niklas Muennighoff, Zitong Yang, Weijia Shi, Xiang Lisa Li, Li Fei-Fei, Hannaneh Hajishirzi, Luke Zettlemoyer, Percy Liang, Emmanuel J. Candès, and Tatsunori Hashimoto. 2025. s1: Simple test-time scaling. *CoRR* abs/2501.19393 (2025). <https://doi.org/10.48550/ARXIV.2501.19393>
- [48] Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. Deep Learning on a Data Diet: Finding Important Examples Early in Training. In *International Conference on Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=Uj7pF-D-YvT>
- [49] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [50] Bashir Rastegarpanah, Krishna Gummadi, and Mark Crovella. 2021. Auditing black-box prediction models for data minimization compliance. *Advances in Neural Information Processing Systems* 34 (2021), 20621–20632.

- [51] Simone Scardapane, Danilo Comminiello, Amir Hussain, and Aurelio Uncini. 2017. Group sparse regularization for deep neural networks. *Neurocomputing* 241 (2017), 81–89.
- [52] Divya Shanmugam, Fernando Diaz, Samira Shabanian, Michele Finck, and Asia Biega. 2022. Learning to Limit Data Collection via Scaling Laws: A Computational Interpretation for the Legal Principle of Data Minimization. In *ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 839–849. <https://dl.acm.org/doi/10.1145/3531146.3533148>
- [53] Ted Shao Wang, Shinan Liu, Jonatas Marques, Nick Feamster, and Sanjay Krishnan. 2025. Algorithmic Data Minimization for Machine Learning over Internet-of-Things Data Streams. *PVLDB* 18, 13 (2025), 5652–5661.
- [54] Saurabh Singh, Sami Abu-El-Haija, Nick Johnston, Jona Ballé, Abhinav Shrivastava, and George Toderici. 2020. End-to-end learning of compressible features. In *IEEE International Conference on Image Processing (ICIP)*. IEEE, 3349–3353.
- [55] Ben Sorscher, Robert Geirhos, Shashank Shekhar, Surya Ganguli, and Ari S. Morcos. 2022. Beyond Neural Scaling Laws: Beating Power Law Scaling via Data Pruning. In *Conference on Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=UmvSLP-PyV>
- [56] Robin Staab, Nikola Jovanović, Mislav Balunović, and Martin Vechev. 2024. From Principle to Practice: Vertical Data Minimization for Machine Learning. In *IEEE Symposium on Security and Privacy (SP)*. 4733–4752. <https://ieeexplore.ieee.org/document/10646763/>
- [57] Robin Staab, Mark Vero, Mislav Balunović, and Martin Vechev. 2023. Beyond memorization: Violating privacy via inference with large language models. *arXiv preprint arXiv:2310.07298* (2023).
- [58] Robert Tibshirani. 1996. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 58, 1 (1996), 267–288.
- [59] Naftali Tishby, Fernando C. Pereira, and William Bialek. 1999. The Information Bottleneck Method. In *Annual Allerton Conference on Communication, Control, and Computing*. 368–377.
- [60] UCI Machine Learning Repository. 1981. Mushroom. <https://doi.org/10.24432/C5959T>
- [61] Antanas Verikas and Marija Bacauskiene. 2002. Feature selection with neural networks. *Pattern recognition letters* 23, 11 (2002), 1323–1335.
- [62] Laura Weidinger, John Mellor, Maribeth Rauh, Conor Griffin, Jonathan Uesato, Po-Sen Huang, Myra Cheng, Mia Glaese, Borja Balle, Atoosa Kasirzadeh, Zac Kenton, Sasha Brown, Will Hawkins, Tom Stepleton, Courtney Biles, Abeba Birhane, Julia Haas, Laura Rimell, Lisa Anne Hendricks, William Isaac, Sean Legassick, Geoffrey Irving, and Iason Gabriel. 2021. Ethical and social risks of harm from language models. *arXiv preprint arXiv:2112.04359* (2021).
- [63] Xiaobo Xia, Jiale Liu, Jun Yu, Xu Shen, Bo Han, and Tongliang Liu. 2023. Moderate Coreset: A Universal Method of Data Selection for Real-world Data-efficient Deep Learning. In *International Conference on Learning Representations (ICLR)*. OpenReview.net.
- [64] Yutaro Yamada, Ofir Lindenbaum, Sahand Negahban, and Yuval Kluger. 2020. Feature selection using stochastic gates. In *International conference on machine learning*. PMLR, 10648–10659.
- [65] Gaoming Yang, Jingzhao Li, Shunxiang Zhang, and Li Yu. 2013. An enhanced l-diversity privacy preservation. In *International Conference on Fuzzy Systems and Knowledge Discovery (FSKD)*. IEEE, 1115–1120.
- [66] Yixin Ye, Zhen Huang, Yang Xiao, Ethan Chern, Shijie Xia, and Pengfei Liu. 2025. LIMO: Less is More for Reasoning. *CoRR* abs/2502.03387 (2025). <https://doi.org/10.48550/ARXIV.2502.03387>
- [67] Hanna Yukhymenko, Robin Staab, Mark Vero, and Martin Vechev. 2024. A Synthetic Dataset for Personal Attribute Inference. In *Conference on Neural Information Processing Systems (NeurIPS)*. <https://openreview.net/forum?id=1nqfQIQBf#discussion>
- [68] Hui Zou and Trevor Hastie. 2005. Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society Series B: Statistical Methodology* 67, 2 (2005), 301–320.

A PROOF OF THEOREM 1

Theorem 1: *The feature-removal case of DMP_ML is NP-hard.* $\square$

Proof: We show that the feature-removal case of DMP_ML is NP-hard by reduction from the NP-complete Set Cover problem (cf. [20]). Given a universe $\mathcal{U} = \{u_1, \dots, u_m\}$, a collection $\mathcal{C} = \{C_1, \dots, C_n\}$ of subsets of $\mathcal{U}$, and an integer K , Set Cover is to determine whether there exists $S \subseteq [1, n]$ such that $|S| \leq K$ and $\bigcup_{j \in S} C_j = \mathcal{U}$. We use base-2 logarithms.

We assume w.l.o.g. that $m \geq 1$ and $1 \leq K < n$. This restriction remains NP-complete: instances with $K = 0$ or no sets can be decided in PTIME and mapped to fixed restricted YES and NO instances; for every other instance, replacing K by $\min\{K, n\}$ and appending an empty set preserves the answer and gives $1 \leq K < n$.

The reduction from Set Cover to the feature-removal case of DMP_ML is given as follows.

(1) Schema $\mathcal{R}$: We use a schema $\mathcal{R} = (A_1, \dots, A_n, A_y)$, where $A_1, \dots, A_n$ are real-valued features and A_y is a binary label. Feature A_j corresponds to set C_j for all $j \in [1, n]$.

(2) Dataset $\mathcal{D}$: We use task tuples to encode the element-set incidences in the Set Cover instance, and independence tuples to encode the number of retained features. More specifically, we define $b_{ij} = 1$ if $u_i \in C_j$, and $b_{ij} = 0$ otherwise. We construct $N = m + n + 1$ tuples. For each $i \in [1, m]$, a task tuple t_i has label 1 and

$$t_i[A_j] = 2i + b_{ij} \quad \text{for all } j \in [1, n].$$

For $j \in [1, n]$, an independence tuple q_j has label 0, $q_j[A_j] = -1$, and $q_j[A_\ell] = 0$ for all $\ell \neq j$. We also add a tuple q_0 with label 0 and value 0 on all features. The empirical distribution is uniform over these N tuples, and accuracy is evaluated under this distribution. Since their raw feature vectors are pairwise distinct, $H(X) = \log N$.

(3) Feature removal ϕ : For any $S \subseteq [1, n]$, ϕ_S removes all features outside S by projecting every feature vector onto $\{A_j \mid j \in S\}$, and retains the label.

(4) Model $\mathcal{M}$ and accuracy bound τ : We use a fixed PTIME training algorithm for $\mathcal{M}$. For every reduced schema, the resulting classifier outputs 1 iff at least one retained coordinate is a positive odd integer, and outputs 0 otherwise. The classifier has a polynomial-size arithmetic description. Let $\tau = 1$. The classifier $\mathcal{M}(\phi_S(\mathcal{D}_x), \mathcal{D}_y)$ correctly classifies all independence tuples and q_0 . For a task tuple t_i , it outputs 1 iff $2i + b_{ij}$ is odd for some $j \in S$, equivalently, iff $u_i \in C_j$ for some $j \in S$. Let $c(S) = |\bigcup_{j \in S} C_j|$. We have

$$\text{acc}(\mathcal{M}(\phi_S(\mathcal{D}_x), \mathcal{D}_y)) = \frac{n + 1 + c(S)}{N}.$$

It follows that the accuracy bound holds iff $\{C_j \mid j \in S\}$ covers $\mathcal{U}$.

(5) Privacy bound ρ : Let $k = |S|$ and $a_k = n - k + 1$. For nonempty S , the task projections are distinct and positive, since the ranges $\{2i, 2i + 1\}$ are disjoint across i . Of the independence tuples, exactly the k tuples q_j with $j \in S$ are distinct and nonzero; all others and q_0 map to zero. Thus, $O_S = \phi_S(X)$ has $m + k$ outcomes of mass $1/N$, and zero has mass a_k/N .

$$H(X \mid O_S) = H(X) - H(O_S) = \frac{a_k}{N} \log a_k =: G_k.$$

The first equality holds since ϕ_S is deterministic. Thus, the task tuples contribute the same m singleton outcomes for every nonempty S , while the independence tuples make G_k depend only

on k . Since $x \log x$ is strictly increasing for $x \geq 1$, G_k is strictly decreasing in k . More specifically, for $k < n$,

$$G_k - G_{k+1} = \frac{a_k \log a_k - (a_k - 1) \log(a_k - 1)}{N} \geq \frac{2}{N},$$

where the numerator is minimized at the integer $a_k = 2$ by the convexity of $x \log x$ on $[1, \infty)$.

We next define the privacy bound. Compute dyadic rationals r_K and r_{K+1} such that $|r_i - G_i| \leq 1/(16N)$ for $i \in \{K, K + 1\}$, and let

$$\rho = \frac{r_K + r_{K+1}}{2}.$$

Binary logarithms of positive integers can be approximated to $O(\log N)$ bits in polynomial time. Hence, r_K , r_{K+1} and ρ have polynomial-bit representations and can be computed in PTIME. The gap above ensures that $G_{K+1} < \rho < G_K$. Therefore, for every nonempty S , $H(X \mid O_S) \geq \rho$ iff $|S| \leq K$. This also holds for $S = \emptyset$, since O_S is constant and $H(X \mid O_S) = H(X) = \log N > G_K > \rho$.

The reduction constructs $N = m + n + 1$ tuples over n features. Its integer values and uniform probability masses use $O(\log N)$ bits, the model is fixed, and the dyadic privacy bound has $O(\log N)$ bits. Thus, the reduction can be constructed in PTIME. We next show that the constructed feature-removal instance is feasible if and only if the Set Cover instance admits a cover of size at most K .

$\Rightarrow$ Assume that there exists a projection ϕ_S that satisfies the accuracy and privacy bounds. By the accuracy construction, $\bigcup_{j \in S} C_j = \mathcal{U}$. By the privacy construction, $|S| \leq K$. Hence, $\{C_j \mid j \in S\}$ is a set cover of size at most K .

$\Leftarrow$ Conversely, assume that $\{C_j \mid j \in S\}$ is a set cover with $|S| \leq K$. The projection ϕ_S has accuracy 1. Moreover, $S \neq \emptyset$ and $H(X \mid O_S) = G_{|S|} \geq G_K > \rho$. Hence, ϕ_S satisfies both bounds.

Therefore, the feature-removal case of DMP_ML is NP-hard. $\square$

B PROOF OF PROPOSITION 2

Proposition 2: *For fixed Σ , k and $\sigma > 0$, with only $W^T W = I$ imposed, the determinant objective in equation (4) can be minimized by the same solution that minimizes the trace objective $\min_{W^T W = I} \text{Tr}(K_O)$. The solution selects the subspace spanned by the k eigenvectors of Σ corresponding to the smallest eigenvalues.* $\square$

Proof: Let $v_1 \leq v_2 \leq \dots \leq v_n$ be the sorted eigenvalues of Σ , and $\mu_1 \leq \dots \leq \mu_k$ the eigenvalues of the projected covariance matrix $W^T \Sigma W$. By Poincaré Separation Theorem [39], for orthonormal W , the projected eigenvalues are bounded by the original eigenvalues:

$$v_i \leq \mu_i \leq v_{n-k+i}.$$

Note that the trace objective can be written as

$$\text{Tr}(K_O) = \sum_{i=1}^k (\mu_i + \sigma^2) \geq \sum_{i=1}^k (v_i + \sigma^2).$$

Equality holds if W spans the eigenvectors corresponding to $\{v_i\}_{i=1}^k$.

The log determinant objective can be similarly bounded as

$$\log \det(K_O) = \sum_{i=1}^k \log(\mu_i + \sigma^2) \geq \sum_{i=1}^k \log(v_i + \sigma^2),$$

since the logarithm is a monotonically increasing function. The equality holds when W spans the eigenvectors corresponding to $\{v_i\}_{i=1}^k$. Therefore, both objectives are minimized by the same W . $\square$

C VARIATIONAL INTERPRETATION OF THE TRACE OBJECTIVE

We derive the leakage bound in the closing remark of Section 6.3, making explicit the connection between the trace objective and the Gaussian variational information bottleneck.

For a fixed learned transformation, write $A = W^T \text{diag}(\hat{g})$, $\mu = \mathbb{E}[AX]$ and $T = \text{Tr}(\text{Cov}(AX))$. The Gaussian channel has conditional density $p(o | x) = \mathcal{N}(Ax, \sigma^2 I_k)$ with $\sigma > 0$. For any reference density r , the variational identity underlying VIB [3] gives

$$\mathbb{E}_X D_{\text{KL}}(p(O | X) \| r) = I(X; O) + D_{\text{KL}}(p_O \| r) \geq I(X; O).$$

Choose $r_{m,v} = \mathcal{N}(m, vI_k)$, where $m \in \mathbb{R}^k$ and $v > 0$. The Gaussian KL formula yields

$$\mathbb{E}_X D_{\text{KL}}(p(O | X) \| r_{m,v}) = \frac{1}{2} \left[\frac{T + \|\mu - m\|^2 + k\sigma^2}{v} - k + k \log \frac{v}{\sigma^2} \right].$$

This is minimized at $m = \mu$ and $v = \sigma^2 + T/k$, giving

$$\min_{m \in \mathbb{R}^k, v > 0} \mathbb{E}_X D_{\text{KL}}(p(O | X) \| r_{m,v}) = \frac{k}{2} \log \left(1 + \frac{T}{k\sigma^2} \right).$$

Thus the bound in Theorem 4 also follows by optimizing an isotropic Gaussian reference distribution. This calculation permits nonzero input means and does not require Gaussian inputs. The reference distribution is used only in the analysis; the release noise remains $\sigma^2 I_k$. For fixed k and σ , minimizing trace minimizes this bound. This correspondence concerns the representation objective, while M2M additionally learns input gates and uses validation accuracy to guide optimization and selection.

D PROOF OF THEOREM 3

Theorem 3: For fixed centered masked data Z , k and σ , and an orthonormal projection matrix W , minimizing the trace of the projected covariance is mathematically equivalent to maximizing the expected squared Frobenius error of the back-projection $\hat{Z}$, that is,

$$\min_{W^T W = I} \text{Tr}(W^T \Sigma W) \iff \max_{W^T W = I} \mathbb{E}_{\mathcal{E}} \|Z - \hat{Z}\|_F^2. \quad \square$$

Proof: Let $P = WW^T$. Since $W^T W = I_k$, P is an orthogonal projector. Then the reconstruction residual can be written as

$$Z - \hat{Z} = Z(I - P) - \mathcal{E}W^T.$$

We first prove that the signal residual $Z(I - P)$ and projection noise $\mathcal{E}W^T$ are orthogonal,

$$\langle Z(I - P), \mathcal{E}W^T \rangle_F = \text{Tr}(W^T (I - P) Z^T \mathcal{E}) = 0,$$

where the equalities hold since trace is cycle-invariant and $W^T (I - P) = W^T - W^T W W^T = 0$.

Therefore, we have

$$\|Z - \hat{Z}\|_F^2 = \|Z(I - P)\|_F^2 + \|\mathcal{E}W^T\|_F^2.$$

For the signal residual term,

$$\begin{aligned} \|Z(I - P)\|_F^2 &= \text{Tr}((I - P)Z^T Z(I - P)) \\ &= \text{Tr}(Z^T Z) - \text{Tr}(W^T Z^T Z W) \\ &= N \text{Tr}(\Sigma) - N \text{Tr}(W^T \Sigma W). \end{aligned}$$

For the noise term,

$$\begin{aligned} \|\mathcal{E}W^T\|_F^2 &= \text{Tr}(W \mathcal{E}^T \mathcal{E} W^T) \\ &= \text{Tr}(\mathcal{E}^T \mathcal{E} W^T W) = \|\mathcal{E}\|_F^2. \end{aligned}$$

Combining these and the fact that $\mathbb{E}_{\mathcal{E}} \|\mathcal{E}\|_F^2 = Nk\sigma^2$, we have

$$\frac{1}{N} \mathbb{E}_{\mathcal{E}} \|Z - \hat{Z}\|_F^2 = \text{Tr}(\Sigma) - \text{Tr}(W^T \Sigma W) + k\sigma^2.$$

Since both $\text{Tr}(\Sigma)$ and $k\sigma^2$ are independent of W , minimizing the projected trace is equivalent to maximizing the expected reconstruction error. $\square$

E DETAILS OF EXAMPLE 7

Consider a dataset $Z \in \mathbb{R}^{4 \times 3}$ and its empirical covariance matrix $\Sigma = \frac{1}{4} Z^T Z$ given as follows:

$$Z = \begin{bmatrix} 3 & 1 & 2 \\ 3 & 1 & -2 \\ -3 & 3 & 0 \\ -3 & -5 & 0 \end{bmatrix}, \quad \Sigma = \begin{bmatrix} 9 & 3 & 0 \\ 3 & 9 & 0 \\ 0 & 0 & 2 \end{bmatrix}.$$

We compute the eigenvalues and eigenvectors of Σ as $v_1 = 2$ ($v_1 = [0, 0, 1]^T$, min variance), $v_2 = 6$ ($v_2 = [1/\sqrt{2}, -1/\sqrt{2}, 0]^T$), and $v_3 = 12$ ($v_3 = [1/\sqrt{2}, 1/\sqrt{2}, 0]^T$, max variance). Here $\mathcal{E}_{ij} \sim \mathcal{N}(0, 0.1^2)$.

PCA. Traditional PCA chooses the eigenvectors with the largest eigenvalues, i.e., $W_{\text{PCA}} = [v_3, v_2]$. This preserves the maximum-variance subspace and yields, for example, the following noisy reconstruction:

$$\hat{Z}_{\text{PCA}} \approx \begin{bmatrix} 2.95 & 1.09 & 0 \\ 3.12 & 0.99 & 0 \\ -3.23 & 2.95 & 0 \\ -3.01 & -4.97 & 0 \end{bmatrix}, \quad \|Z - \hat{Z}_{\text{PCA}}\|_F^2 \approx 8.08.$$

Trace minimization. In contrast, trace minimization prioritizes the smallest-eigenvalue directions. When utility constraints are not imposed, M2M selects $W_{\text{M2M}} = [v_1, v_2]$, which discards more variance and yields the larger noisy reconstruction error:

$$\hat{Z}_{\text{M2M}} \approx \begin{bmatrix} 0.93 & -0.93 & 2.03 \\ 1.07 & -1.07 & -1.93 \\ -3.09 & 3.09 & -0.20 \\ 0.98 & -0.98 & 0.01 \end{bmatrix}, \quad \|Z - \hat{Z}_{\text{M2M}}\|_F^2 \approx 48.08.$$

The corresponding expected squared back-projection errors are $4(20 - 18 + 0.02) = 8.08$ and $4(20 - 8 + 0.02) = 48.08$, respectively. This example illustrates Theorem 3: selecting low-variance directions increases expected squared back-projection error.

F PROOF OF THEOREM 4

Theorem 4: Fix a feasible (s^*, W^*, θ^*) for (6). For empirical X , let $O = (W^*)^T (\hat{g}(s^*) \odot X) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2 I_k)$ is independent of X and $\sigma > 0$. Then:

$$\begin{aligned} H(X | O) &\geq H(X) - \frac{k}{2} \log \left(1 + \frac{1}{k\sigma^2} \sum_{j=1}^n v_j P_j(W^*) \right), \\ \text{acc}(\mathcal{M}_{\theta^*, s^*, W^*}) &\geq \tau. \end{aligned} \quad \square$$

Proof: The utility guarantee directly follows from the optimization constraints. For the privacy gain, by the definition of conditional entropy, we have

$$H(X|O) = H(X) - I(X; O).$$

We next upper-bound the mutual information $I(X; O)$. By the Gaussian maximum-entropy theorem in Section 6.2, we obtain

$$h(O) \leq \frac{1}{2} \log \det(2\pi e K_O).$$

Here $K_O = (W^*)^T \Sigma_Z W^* + \sigma^2 I_k$ is the covariance matrix of the smoothed output O , with $\sigma > 0$. Since the noise ϵ is independent of the input data X , the conditional entropy $h(O | X)$ is

$$h(O|X) = h(\epsilon) = \frac{1}{2} \log \det(2\pi e \sigma^2 I_k).$$

The mutual information is therefore upper-bounded by

$$\begin{aligned} I(X; O) &= h(O) - h(O|X) \\ &\leq \frac{1}{2} \ln \left(\frac{\det((W^*)^T \Sigma_Z W^* + \sigma^2 I_k)}{\det(\sigma^2 I_k)} \right) \\ &= \frac{1}{2} \ln \det \left(I_k + \frac{1}{\sigma^2} (W^*)^T \Sigma_Z W^* \right). \end{aligned}$$

Let $\mu_1, \dots, \mu_k$ be the eigenvalues of $B = (W^*)^T \Sigma_Z W^*$. Then

$$\log \det \left(I_k + \frac{1}{\sigma^2} B \right) = \sum_{i=1}^k \log \left(1 + \frac{\mu_i}{\sigma^2} \right).$$

By the concavity of $\log(1+x)$ and Jensen's inequality, we have

$$\frac{1}{k} \sum_{i=1}^k \log \left(1 + \frac{\mu_i}{\sigma^2} \right) \leq \log \left(1 + \frac{1}{k} \sum_{i=1}^k \frac{\mu_i}{\sigma^2} \right).$$

Since $\sum_{i=1}^k \mu_i = \text{Tr}(B)$, we can multiply both sides of the inequality above by k and obtain

$$\log \det \left(I_k + \frac{1}{\sigma^2} B \right) \leq k \log \left(1 + \frac{\text{Tr}(B)}{k\sigma^2} \right).$$

Finally, we decompose the projected trace in the direction of eigenvectors of Σ_Z . In particular, we can write Σ_Z as

$$\Sigma_Z = \sum_{j=1}^n v_j u_j u_j^T,$$

where v_j and u_j are the corresponding eigenvalue and eigenvector, respectively. Substituting the spectral decomposition into the trace of the projected space, we obtain

$$\begin{aligned} \text{Tr}(B) &= \text{Tr} \left((W^*)^T \Sigma_Z W^* \right) = \text{Tr} \left((W^*)^T \left(\sum_{j=1}^n v_j u_j u_j^T \right) W^* \right) \\ &= \sum_{j=1}^n v_j \text{Tr} \left((W^*)^T u_j u_j^T W^* \right) \\ &= \sum_{j=1}^n v_j \text{Tr} \left(u_j^T W^* (W^*)^T u_j \right) \\ &= \sum_{j=1}^n v_j P_j(W^*), \end{aligned}$$

where $P_j(W^*) = \|(W^*)^T u_j\|^2 = \text{Tr} \left(u_j^T W^* (W^*)^T u_j \right)$.

Combining these inequalities, we obtain

$$\begin{aligned} H(X|O) &= H(X) - I(X; O) \\ &\geq H(X) - \frac{k}{2} \log \left(1 + \frac{1}{k\sigma^2} \sum_{j=1}^n v_j P_j(W^*) \right). \end{aligned}$$

□

G PROOF OF THE RETAINED-LABEL BOUND

We prove the bound on $H(X | O, Y)$ stated in the *Retained labels* paragraph of Section 7.1.

Fix the transformation in Theorem 4 and let (X, Y) follow the empirical joint distribution of labeled training rows. Since release noise is independent of (X, Y) and the feature transformation uses only X , we have $I(Y; O | X) = 0$. We apply the chain rule twice:

$$\begin{aligned} I(X; O) &= I(X, Y; O) \\ &= I(Y; O) + I(X; O | Y). \end{aligned}$$

Nonnegativity of $I(Y; O)$ and Theorem 4 imply $I(X; O | Y) \leq B_{\text{tr}}$. Hence

$$H(X | O, Y) = H(X | Y) - I(X; O | Y) \geq H(X | Y) - B_{\text{tr}}.$$

Taking the maximum with zero yields the stated bound. The baseline $H(X | Y)$ accounts for information already revealed by the label; in particular, this result claims no protection for a feature determined by Y .

H ADDITIONAL EXPERIMENTAL RESULTS

We give experimental details and additional results for Section 8.

Experimental setting. We specify data generation, evaluation protocols and selected configurations.

Datasets. For categorical synthetic data, every input attribute is binary. We divide the attributes equally into high- and low-entropy groups. High-entropy attributes satisfy $\Pr[X_i = +1] = 0.5$, whereas low-entropy attributes satisfy $\Pr[X_i = +1] = 0.9$ by default. All attributes are sampled independently. We standardize each attribute using its training-set mean and variance, and generate labels using a fixed teacher model over all standardized attributes. The coefficients have equal magnitude and independently sampled signs, giving each attribute a comparable contribution to the label. The preceding standardization specifies the teacher inputs used for label generation. The inputs supplied to the minimizer and classifier use the same per-attribute standardization, with preprocessing fitted on the training split only.

Metrics. The reconstruction target $r(x)$ has d_{rec} entries, one per original input attribute. Categorical predictions use $\arg \max$ over the predicted encoded columns of each attribute. The majority predictor uses the same target units, with majority values fitted on the raw training split.

For linkage, we use a fixed seed-specific procedure. The training split provides aligned raw-release auxiliary pairs. For each of ten candidate pools, we sample 200 unique test profiles without replacement and randomly permute them. Methods share raw auxiliary records and candidate sets.

Configurable parameters. Table 4 reports the selected settings for the main comparison, shared by all three runs. The split column gives the training/validation/test ratio; preprocessing follows Section 8. The parameter sweeps use separate settings, detailed below.

Configuration. We report the sample standard deviation over three runs, computed with denominator 3 – 1. Each repeat uses a distinct seed for model initialization, minibatch order, gate sampling and attacker training. Release noise uses a separate seeded generator; noise draws are not reused across records or splits. The data split

Dataset	Experiment	k	σ	η	x	Split	acc
CoverType	Main	8	0	0.2	80	60/20/20	83.52
Adult	Main	2	0.05	0.2	50	60/20/20	81.82
Mushroom	Main	2	0.25	0.2	80	60/20/20	91.30
NATICUSdroid	Main	2	0.05	0.2	80	60/20/20	89.49
Marketing	Main	2	0.05	0.2	80	60/20/20	76.47

Table 4: Selected configurations: mean test accuracy (%).

Dataset	M2M	PAT	MetaModeling	ANOVA	Representative
Test accuracy					
CoverType	83.52 $\pm$ 0.34	84.15 $\pm$ 0.27	87.57 $\pm$ 0.79	82.29 $\pm$ 0.22	–
Adult	81.82 $\pm$ 0.53	81.83 $\pm$ 0.32	82.36 $\pm$ 0.23	82.06 $\pm$ 0.26	82.19 $\pm$ 0.25
Mushroom	91.30 $\pm$ 0.51	98.50 $\pm$ 0.26	98.50 $\pm$ 0.26	98.50 $\pm$ 0.26	99.34 $\pm$ 0.98
NATICUSdroid	89.49 $\pm$ 0.78	92.30 $\pm$ 0.40	92.92 $\pm$ 0.30	91.85 $\pm$ 0.34	96.28 $\pm$ 0.45
Marketing	76.47 $\pm$ 0.95	77.47 $\pm$ 0.50	76.47 $\pm$ 1.78	76.89 $\pm$ 1.30	76.65 $\pm$ 1.17
Normalized reconstruction risk					
CoverType	48.17 $\pm$ 2.89	62.75 $\pm$ 9.39	89.26 $\pm$ 4.02	70.29 $\pm$ 3.48	–
Adult	11.66 $\pm$ 0.36	18.31 $\pm$ 2.43	47.72 $\pm$ 11.92	41.67 $\pm$ 0.16	26.44 $\pm$ 0.49
Mushroom	7.46 $\pm$ 1.76	9.45 $\pm$ 2.61	26.74 $\pm$ 1.16	26.74 $\pm$ 1.16	66.63 $\pm$ 20.89
NATICUSdroid	18.19 $\pm$ 5.99	28.51 $\pm$ 8.74	51.77 $\pm$ 16.38	32.14 $\pm$ 8.21	64.67 $\pm$ 5.86
Marketing	18.20 $\pm$ 0.82	32.77 $\pm$ 4.33	45.57 $\pm$ 4.55	46.23 $\pm$ 5.26	60.08 $\pm$ 19.60

Table 5: Main comparisons: mean $\pm$ sample standard deviation over three runs, in percent. A dash denotes the absence of a validation-feasible result within the stated budget.

Dataset	Removed	Partial	Retained
CoverType	0.7	52.3	1.0
Adult	4.0	9.7	0.3
Mushroom	1.0	11.0	10.0
NATICUSdroid	42.0	0.0	44.0
Marketing	0.0	8.0	5.0

Table 6: Original-attribute retention for the main configurations. The three categories partition the source attributes.

across repeats is a 60/20/20 training/validation/test partition.

Exp-1: Effectiveness. We report variability, attribute-retention statistics and leakage bounds, and detail the parameter sweeps.

Normalized reconstruction risk vs. baselines. Table 5 reports test accuracy and normalized reconstruction risk for the selected configurations in the main comparison, as mean $\pm$ sample standard deviation over three runs. The values supplement the means in Section 8; the observed variation in the method differences is small relative to the differences between methods and does not alter their overall ranking.

Feature-level interpretability. Let G_j contain the encoded columns of original attribute j . It is removed if all gates in G_j are zero, partially gated if some are zero, and retained otherwise. Table 6 uses the deterministic gates before rounding. Partial one-hot gating can leave the original category uniquely identifiable, for example when only one indicator is removed. These counts describe input retention. An attribute can be omitted from acquisition only when the source supplies retained inputs without its value; central pre-processing of that value instead reduces retention after acquisition.

Trace and reconstruction. For Figure 3(d), we add another trace weight $\lambda_{tr} \in \{0, 0.1, 0.5, 1, 2, 4, 8, 16\}$ on the trace penalty. k and

σ are fixed to 2 and 0.25 respectively; the shared data and preprocessing are held fixed across trace weights within each seed. The trace normalization is $\text{Tr}(W^\top \Sigma_g W) / \text{Tr}(\Sigma_g)$, and the reported error uses hard one-hot argmax reconstruction with one fixed noisy release.

Empirical leakage bounds. For each selected run, fix the learned mask and projection, and let $r_i = W^\top (\hat{g}(s) \odot x_i)$ and $\bar{r} = N^{-1} \sum_i r_i$. We compute the absolute projected trace $T = N^{-1} \sum_i \|r_i - \bar{r}\|_2^2$ on the empirical training distribution. For the release noise scale $\sigma > 0$, Theorem 4 yields

$$B_{tr} = \frac{k}{2} \log \left(1 + \frac{T}{k\sigma^2} \right).$$

This uses the final deterministic gate and release noise scale, rather than the stochastic training penalty or percentage trace reduction. Let $H_{emp}(X) = -\sum_v p_v \log p_v$, where p_v is the frequency of training feature vector v . We compute the residual-uncertainty bounds

$$L = \max\{0, H_{emp}(X) - B_{tr}\},$$

$$L_Y = \max\{0, H_{emp}(X | Y) - B_{tr}\},$$

where $H_{emp}(X | Y) = \sum_y p_y H_{emp}(X | Y = y)$ is computed by grouping labeled training rows. These bound uncertainty about the full input vector without and with retained labels, respectively, under the empirical distribution; they are not population estimates.

Table 7 reports each dataset’s bound ranges over selected positive-noise runs. Ranges include zero bounds; “n/a” denotes no positive-noise run. A zero lower bound does not establish positive residual uncertainty; attacks assess reconstruction risk separately.

Optimizer and adjustment settings. We varied one setting at a time, using the defaults from Section 8 unless stated otherwise and the

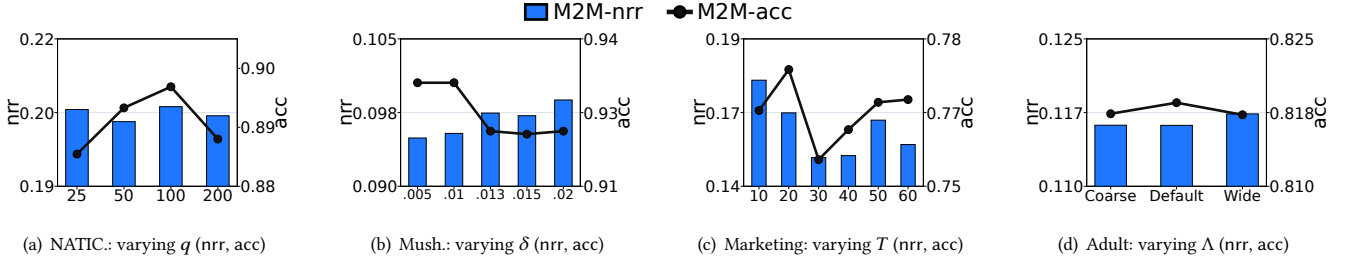


Figure 4: Sensitivity to M2M_AGDA adjustment settings

Dataset	Noisy runs	Max. shortfall (pp)		Bound range (nats)	
		Validation	Test	L	L_Y
CoverType	0	0	0	n/a	n/a
Adult	3	0	0	[8.37, 8.74]	[7.88, 8.24]
Mushroom	3	0	0	[7.71, 7.83]	[7.02, 7.14]
NATICUSdroid	3	0	0	[5.51, 5.57]	[4.86, 4.92]
Marketing	3	0	0	[5.77, 6.27]	[5.11, 5.61]

Table 7: Utility shortfalls and empirical residual-uncertainty bounds for the main M2M configurations. Shortfalls summarize all selected runs; validation shortfalls are zero by selection. “Noisy runs” counts those with $\sigma > 0$, used for the bound ranges.

same validation-based selection. We report test nrr and acc.

We compared SGD [11], Adam [29], AdamW [37], and RMSprop [26] as the primal optimizer OPT of AGDA on Mushroom, fixing all other M2M and AGDA settings. All optimizers find utility-feasible solutions, with average nrr ranging from 6.72% to 8.75%. Each solution releases $k = 2$ aggregated features; selected runs retain 89.7 encoded features on average across optimizers. RMSprop achieves 23.20% lower average nrr than SGD (not shown).

(1) *Adjustment interval q .* We varied q over $\{25, 50, 100, 200\}$ on NATICUSdroid, while fixing the total number Tq of primal updates. As shown in Figure 4(a), M2M_AGDA reaches the validation utility threshold in all four settings; nrr ranges from 20.32% to 20.62%, and acc from 88.55% to 89.69%. Neither metric exhibits a consistent monotonic trend, since q controls how long the primal variables respond to a fixed multiplier and how often validation adjusts it.

(2) *Accuracy margin δ .* We varied δ over $\{0.005, 0.01, 0.013, 0.015, 0.02\}$ on Mushroom, fixing the other settings. As shown in Figure 4(b), nrr ranges from 9.49% to 9.88%, while acc ranges from 92.06% to 93.11% and remains above τ in all five settings. The minimum nrr is 3.92% lower than the maximum, with no consistent monotonic trend.

(3) *Outer-iteration budget T .* We varied T from 10 to 60 on Marketing, keeping $q = 100$ fixed. As shown in Figure 4(c), nrr generally decreases as T increases to 40, but fluctuates thereafter, with no consistent improvement from a larger budget. Across the tested values, nrr ranges from 14.97% to 17.60%, with the minimum 14.92% lower than the maximum, while acc ranges from 75.54% to 77.38%. End-to-end runtime increases from 20.92 s at $T = 10$ to 111.27 s at $T = 60$.

(4) *Multiplier grid Λ .* We tested 3 grids with different ranges and granularities on Adult. As shown in Figure 4(d), across grids that yield validation-feasible solutions, the minimum nrr is 0.99% lower than the maximum, and the maximum acc is 0.15% higher than the minimum. The differences are non-monotonic across the tested

grids, showing that M2M_AGDA does not rely on a particular grid resolution within the tested ranges.

Impact of k . The main CoverType configuration in Table 4 and the dimension sweep use different fixed settings; the main configuration uses validation-constrained M2M_AGDA with an 80% retained-accuracy target, whereas the sweep uses a fixed multiplier $\lambda = 1$. All other training, noise, regularization, attack, data, and preprocessing settings are unchanged. Within the sweep, only k varies.

Impact of σ . For the NATICUSdroid noise sweep, we fix $k = 2$, $\eta = 0.2$, the utility multiplier to $\lambda = 1$ and the classifier to a 256×256 MLP. We use the same stratified 60/20/20 split and preprocessing as the main comparison; the only differences are fixed-multiplier training and varying σ .

Exp-3: Ablation. We specify the component comparisons and synthetic data used in the ablation study.

Component ablation. The no-trace variant retains the hard-concrete gates, entropy weights, sparsity penalty, orthonormal projection and task classifier. Its training objective removes only the projected-trace term; the gates, projection and classifier remain trainable.

For each paired run, we use the same preprocessing, split, initialization seed, k , σ , η , utility threshold and training budget. Both share Adam, the same accuracy-guided multiplier grid and the same configuration-selection budget. Full M2M selects the validation-feasible checkpoint minimizing $f(s, W)$, whereas the no-trace variant applies the same rule after setting the trace component to zero. The retained-accuracy target is 50% for Adult and 80% for Mushroom and NATICUSdroid; σ is 0.25 for Mushroom and 0.05 otherwise. Testing follows selection.

The aggregation-only variant retains all inputs and uses the same trace objective and orthogonal linear representation family. For the full/aggregation-only comparison, each pair uses the same seed-specific stratified 60/20/20 split for seeds 0, 1 and 2. Preprocessing, k , σ , classifier, optimizer, multiplier grid and selection protocol are

Variant	Adult		Mushroom		NATICUSdroid	
	acc	nrr	acc	nrr	acc	nrr
M2M	81.78 $\pm$ 0.35	11.48 $\pm$ 0.22	89.54 $\pm$ 0.83	5.84 $\pm$ 1.73	89.09 $\pm$ 0.67	19.49 $\pm$ 8.08
M2M _{sel}	81.67 $\pm$ 0.41	30.27 $\pm$ 4.83	99.77 $\pm$ 0.25	50.19 $\pm$ 0.36	93.64 $\pm$ 0.25	43.34 $\pm$ 5.76
M2M _{agg}	82.27 $\pm$ 0.43	12.05 $\pm$ 0.78	94.73 $\pm$ 0.40	7.59 $\pm$ 2.31	93.87 $\pm$ 0.24	23.19 $\pm$ 7.09
M2M _{PCA}	81.67 $\pm$ 0.41	16.88 $\pm$ 4.41	99.77 $\pm$ 0.25	33.97 $\pm$ 4.75	93.64 $\pm$ 0.25	33.20 $\pm$ 6.42
M2M _{noTr}	82.48 $\pm$ 0.32	21.88 $\pm$ 0.64	95.20 $\pm$ 0.59	18.08 $\pm$ 3.79	92.95 $\pm$ 0.55	33.75 $\pm$ 7.14

Table 8: Component ablation: test accuracy and NRR, as mean $\pm$ standard deviation over three runs, in percent. The no-trace comparison keeps the representation family fixed; the variant omits trace from both training and checkpoint selection.

identical within each pair.

Table 8 reports test accuracy and normalized reconstruction risk for full M2M and its component variants, as means and sample standard deviations over three runs. On Adult, Mushroom and NATICUSdroid, respectively, the mean NRR differences (no-trace minus full) are 10.40, 12.24 and 14.26 percentage points. The corresponding mean test-accuracy differences are 0.70, 5.66 and 3.86 percentage points. For aggregation-only minus full, the mean NRR differences are 0.57, 1.75 and 3.70 percentage points, and the mean test-accuracy differences are 0.49, 5.19 and 4.78 percentage points, respectively. Thus, full M2M has lower mean reconstruction risk and lower mean test accuracy than both variants on all three datasets. The reported standard deviations show variability across runs for each method.

For the main configurations in Table 6, full M2M removes an average of 4.0, 1.0 and 42 original attributes on the three datasets, respectively, using the all-indicators-zero definition.

Synthetic ground-truth validation. The dataset has $|\mathcal{D}| = 200k$ and $n = 200$. The uniform-weight variant sets $w_i = \bar{w}$ for all features,

where $\bar{w} = \frac{1}{n} \sum_{j=1}^n w_j$. Averaged over three seeds at the same utility target, M2M and M2M_{unif} retain 50.67% and 31.34% of features, respectively. Their removal rates are 98.67% and 65.00% for high-entropy features, and 0% and 72.33% for low-entropy features.

Mixed-type robustness. For mixed-type data, half the attributes are continuous and sampled independently from $\mathcal{N}(0, 1)$. Of these, 50 are task-relevant and 50 are irrelevant; labels depend only on the relevant attributes. We use 60/20/20 training/validation/test splits. The teacher receives raw relevant attributes, while the minimizer and classifier receive attributes standardized using training-split means and standard deviations. Reconstruction predictions are inverse-transformed, and NRMSE is computed on the raw continuous scale, where the constant training-mean predictor has asymptotic NRMSE 1.

Ablation of M2M_AGDA. For Figure 3(q), the multiplier grid is the same as the main configuration, with $|\Lambda| = 13$. Thus, grid search has $|\Lambda|$ times the total update budget of M2M_AGDA. The default grid in Section 8 has 13 values.